



AFRICAN INSTITUTE OF COMPUTER SCIENCES (AICS)

BP: 13719

TEL: (+237) 242 72 99 57 / 242 72 99 58

WEB: www.iaicameroun.com

E-MAIL: contact@iaicameroun.com

UML

Mr MESSIO

Computer Engineer, Master in IS's Management / Teacher in IAI Cameroon

Email: robbenmessio@gmail.com

MR. MESSIO, Computer Engineer, Master Degree in IS's Management / Lecturer in AICS

MODELING INFORMATION SYSTEM: OBJECT APPROACH WITH UML

CHAPTER I: INFORMATION SYSTEMS

I. DEFINITION

An information system is the central node of a company or organization. Its main role is to enable the various actors to convey information and communicate it thanks to a set of material, human and software resources.

In summary, an information system makes possible to create, collect, store and process information within an organization.

We can't talk about an information system without really understanding the following two basic concepts: data and information.

II. DIFFERENCE BETWEEN DATA AND INFORMATION

Data is a raw value that has not yet been analyzed or interpreted. Taken out of context, it cannot be understood. In short, a piece of data has no real meaning.

Example: the code X13 does not mean anything to an average user, but it does represent the code of a staff whose name is Messio in the company IAI. (It can also be the number of a sales order, an invoice, etc.).

Information, on the other hand, is data that is processed and put into context in order to be understandable by a user.

Data is used as inputs, which must be processed and organized in a particular way to generate output, i.e., **information**.

III. THE QUALITIES OF GOOD INFORMATION

A piece of information must have the following attributes:

- **Availability:** always available when needed and not expired when available.
- **Completeness:** includes everything the user needs to know about the situation in which it is being used.
- **Brevity:** does not include unnecessary elements
- **Significance:** Understands only what is important in the current situation
- **Accuracy:** corresponds to the reality it represents, does not contain errors
- **Accuracy:** Provides quantitative information with the appropriate degree of accuracy to the corresponding data.

IV. THE QUALITIES OF GOOD INFORMATION

An organization can be thought of as a structured system. A system is a set of elements connected by a set of relationships. It is characterized by:

- Different elements that make it up: these can be tangible elements (premises, machines, etc.), human elements (employees), financial elements (capital, etc.)
- A network of interactions that allows the exchange of capital or information in a company is divided into 3 main systems: the operating system, the information system and the management system.

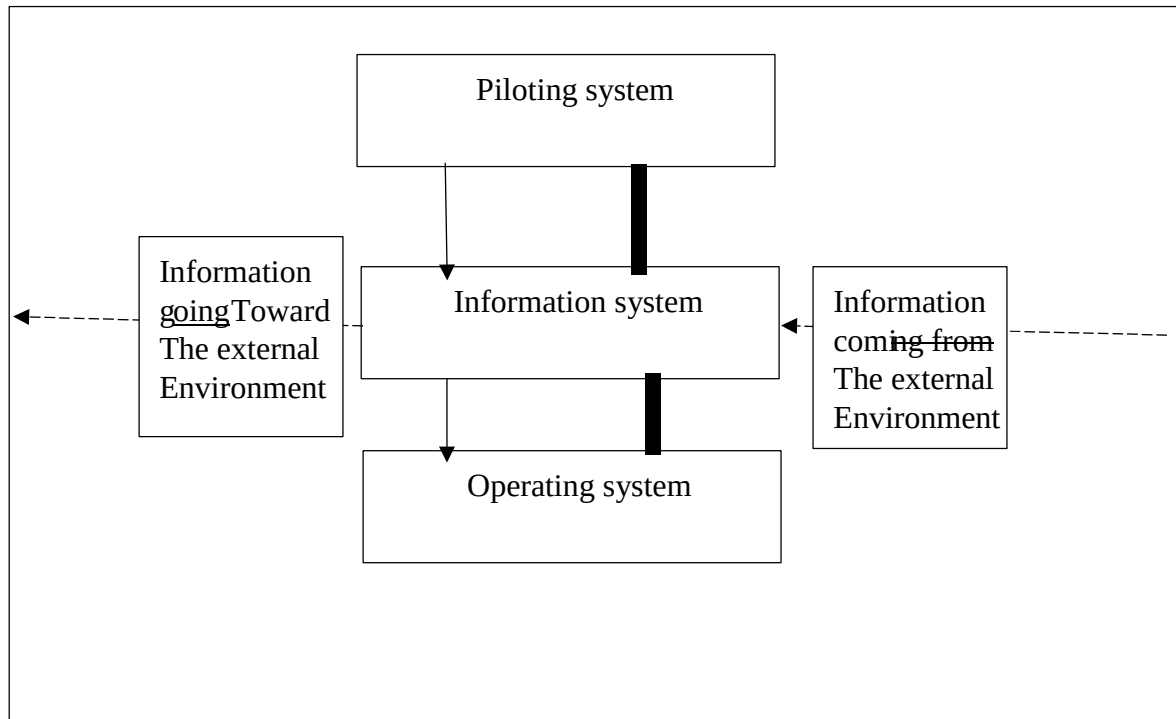


Fig: Sub-systems of an IS

- **The information system** serves as a means of communication between the company, the external environment and the operating system and the control system.
- **The management system:** is the one that controls and steers the operating system, it is at the head of the organization setting objectives and making decisions.
- **The operating system:** this is what the basis of any organization is, it is this system that allows the transformation of information whose objective is to return it to the right person. It corresponds to the different departments of a company. He is in charge of the day-to-day management of the company: personnel management, production management, stock management, accounting, invoicing, etc.

The difference between IS, computerized system and computer system

Computer system involves the design and development of all types of software from operating systems, it focuses on programming, data structures, and operating systems, while information systems focus on how data is gathered, stored, and transformed into information. A computerized system, process, or business is one in which the work is done by computer.

V. THE FUNCTIONS OF AN INFORMATION SYSTEM

There are therefore 4 main functions of an IS:

- **Collect:** this is where the data is born, whether you acquire information from the internal or external environment of the company.
- **Store:** as soon as the information is acquired, the information system stores it (either on files or in a database). It must be available and must be able to be preserved over time.
- **Transform/Process:** Processing or transformation consists first of all in accessing the information, then in organizing the information, i.e.: structuring the information according to specific criteria, for example: grouping the information by customer, by geographical area, the processing also consists of updating the information and in order to produce new information based on the existing information.
- **Disseminate:** the IS then transmits the information to its internal or external environment.

The objective of the Information System is therefore to provide information within an organization that can be directly used by the various stakeholders and facilitate decision-making. All this shows the importance of computerizing the information systems of our companies. Because a computerized information system makes it possible to increase productivity, improve the working conditions of employees, improve the quality of products or services, etc.

VI. THE COMPONENTS OF AN INFORMATION SYSTEM

- **Hardware:** servers, computers, printers, scanners, firewalls, routers, etc.
- **Software:** system programs, application programs, security programs, etc.
- **Databases:** Curated collections of data used by application software
- **Telecommunications:** intranet, extranet, internet
- **Human resources:** IT engineers, IS managers, IS users
- **Procedures:** user specifications, operations, maintenance, online help, user manuals, operator manuals, other documents often in digital form.

CHAPTER II: UML

The Unified Modelling Language (UML) is a consolidation of the best Unified Modelling practices that have been established over the years in the use of modelling languages. UML enables us to present the widely varying aspects of a software system (e.g., requirements, data structures, data flows, and information flows) within a single framework using object-oriented concepts. Before we venture too deeply into UML, however, in this chapter we first explain why modelling is an indispensable part of software development. To do this, we look at what models are and what we need them for. We briefly recapitulate the basic concepts of object orientation before continuing with an overview of the structure of the book.

I. SCENARIO

Imagine that you want to develop a software system that a customer has ordered from you. One of the first challenges you are confronted with is clarifying what the customer actually wants and whether you have understood the customer's exact requirements for the prospective system. This first step is already critical for the success or failure of your project. The question is, ***how do you communicate with your customer?*** Natural language is not necessarily a good choice as it is imprecise and ambiguous. Misunderstandings can easily arise and there is a very great risk that people with different backgrounds (e.g., a computer scientist and a business manager) will talk at cross-purposes, which can have serious consequences. What you need is to be able to **create a model for your software**. This model highlights the important aspects of the software in a clear form of notation that is as simple as possible but abstracts from irrelevant details, just like models in architecture, e.g., construction plans. A construction plan for a building contains information such as the floor plan. Construction materials to be used are not specified at this point in time; they are irrelevant and would make the plan more complicated than necessary. The plan also does not contain any information about how the electrical cables are to be laid. A separate plan is created for this aspect to avoid presenting too much information at once. Just like in architecture, it is important in information technology that people with different backgrounds (e.g., architect and builder) can read, interpret, and implement the model.

I.1. Modelling languages

Modelling languages were developed precisely for such scenarios and demonstrate clearly defined rules for a structured description of a system. These languages can be textual (e.g., a programming language such as Java) or visual (e.g., a language that provides symbols for transistors, diodes, etc. that can be combined with one another). Modelling languages can be designed for a specific domain, for example, for describing web applications. On the one hand, these domain-specific modelling languages provide tools and guidelines for solving problems in a specific field efficiently; on the other hand, they can also be restrictive. Alternatively, modelling languages can be designed for general purpose use. The language UML, which is the subject of this book, is a general purpose modelling language. We will use UML to get to know the most important concepts of object-oriented modelling.

I.2. Object-oriented

Object-oriented modelling is a form of modelling that obeys the modelling object-oriented paradigm. In the following two subsections, we will look briefly at the notion of a model and the main concepts of object orientation. This will provide us with a good basis for our subsequent examination of object-oriented modelling with UML.

1.3 Models

Models allow us to describe systems efficiently and elegantly. A system is an integrated whole made up of components that are related to one another and influence each other in such a way that they can be perceived as a single, task-based or purpose-based unit. In this regard, they separate themselves from the surrounding environment. Examples of systems are material things, such as cars or airplanes, ecological environments, such as lakes and forests, but also organizational units such as a university or a company. In information technology, we are interested in Software systems in particular in software systems and thus in models that describe software systems

It is also a good way to control its complexity and ensure its consistency, the purpose of computer modelling is to be able to bring out quality software that meets the following criteria

- **Validity:** it is the ability of a software product to perform exactly its function defined by the specification or requirement of the user.
- **Reliability:** it is the ability of a software product to operate regardless of the operating conditions (component failure, load increase).
- **Maintainability/Extensibility:** It is the ease of making correction or adding function to a software

- **Reusable:** it is the ability of a software to be reuse in role or in path in new application
- **Compatibility:** It is the ease with which a software can be combine with other software
- **Portability:** It is the ability of a software to operate on many platforms.
- **Integrity:** it is the ability of a software to protect its own data against unauthorised access.
- **Easy to use:** easy to learn, use, prepare data, interpret error and recover from users errors.

We model a system to visually and graphically describe the needs and the functional and technical solutions of a software project.

I.4 Life cycle of a software

In order to solve the problem that cause the software crises in the 1970's mainly due to

- o Increase cost
- o Maintenance and development difficulties
- o Unreliability
- o Non-respect of the specifications
- o Non-respect of the deadline

Software proposes a set of steps which if respected lead to quality software called the **software life cycle**. The life cycle design all the stages of software development from its conception to its disappearance. The stages of the software life cycle are as follows

- Preliminary study (feasibility study)
- The elaboration of specification or expression of it
- Analysis
- Design
- Coding
- Test
- Maintenance

I.5 History of UML

From the late 1980s to 1990s many methods or language for representing OOP were developed and implemented. To unify these languages, 3 developers; **James Rum Baugh, Grady Brooch, Ivar Jacobson** decided to merge several existing languages (the **Brooch method, the object modelling technique (OMT), the object-oriented software engineering method (OOSE)**) into a language which is common and standardize called **UML**.

UML (Unified Modelling Language) is a graphical modelling language based on schema called **diagrams** design to provide a standard visualization when designing a software. It was in 1997 that the OMG (Object Management Group which is a consortium and which goal is to promote and standardize object oriented) developed UML 1.0 due to dissatisfaction, the developers settled up a working group to improve the language on several versions. A major religion was then carried out and we left from **UML 1.0** to **UML 1.5** which include 9 diagrams.

Today the latest version of the specification validated by the OMG in 2017 is **UML 2.5.1** (which bring radical innovation and greatly extend the scope of UML) and which include 14 diagrams.

NB: UML does not recommend any approach so it is not a method. Everyone is free to use the diagram type they wants in the order they wants. It is enough if the diagram produced are consistent with each other before moving on to the realisation of the software.

I.6. Object Approach

UML is an object-oriented language i.e. it considers the software as a collection of dissociated and identified object with characteristics or attributes (attributes are variable storing information about the object) and method (the set of action or operation that can be perform by the object).The particularity of object-oriented is that it bring data and treatment closer.

This approach is organised around 3 main functions

- *It is iterative and incremental:* it consist of going back and forth between the initial plan and the modification made by the project stakeholder.
- *Driven by customer and user name:* this need to represent the customer requirement that would have to be implemented and that would serve as a common thread throughout the development cycle.
- *Centred on the software architecture:* the software architecture describes the strategic choices which largely determine the quality of the software (performance, reliability, adaptability, etc). We can state the client state architecture which is in layer or levels. This involve asking basic question like;

- What would be the different software component to use (database, etc.).
- Which hardware will each of the component be installed on?

Some important notions are specific to the object approach and contributes to the implementation of quality software.

i. **The class:**

It is an abstract data type which specify the characteristics (attributes and methods) common to a whole family of object and which make possible to create object having this characteristics. In other word a class declare property common to a set of objects and it is a mould from which it is possible to create objects.

Example: The class animal has as attribute name, specie, sex, type, etc. with method shout (), move ().

ii. **Encapsulation**

It consist of hiding an implementation of an object by defining an interface, this mean that the user of a class doesn't necessary have to know how the data has been structured. It guarantees the integrity of the data because it makes possible to prohibited or restrict the direct access to the attribute of the object. It define the levels of visibility (access right on data). We have 3 main levels of visibility:

- Public

The function of all classes can access data or method of a class defined with the public visibility level. This is the lowest level of data protection.

- Protected

Here access to data is reserved for the function of the inheriting classes i.e. by the member functions of the class as well as the derived classes.

- Private

Here access to data is limited to the method of a class itself. This is the highest level of data protection.

iii. **Inheritance:** it is a mechanism for transmitting the characteristics of a class (attribute or method) to a sub class. A class can be specialized in order to add specific characteristics or to adapt some of them. Several classes can be generalized into a class in order to group with common characteristics of a set of classes. Thus, specialization and generalisation make it possible to build class hierarchy. Inheritance can be single (JAVA, PHP) or multiple (C++, python).

- iv. **Polymorphism:** It represent the ability of a method to apply to object of different classes. Polymorphism therefore increases the quality of the codes.

UML is composed of several parts:

- **The views:** They are the observable part of the system they describes from a given point of view i.e. organizational, dynamic, temporal, architectural, geographical, logical, etc. When combining all these views it is possible to define a complete system.
- **The diagrams:** They are graphical elements which describes the content of views. They are abstract notions.

II. The UP (Unified Process)

The mastery of development processes involves an organization and follow-up of activities. UP (Unified Process) is a generic software development method. Generic means that it is necessary to adapt UP to the context of the project, team, domain and/or organization.

1. Characteristics of UP

The unified process is a software development process: it brings together the activities to be carried out to transform the needs of a user into a software system. The essential characteristics of the unified process are:

- It is component-based,
- It uses the UML language (set of tools and diagrams),
- It is driven by use cases,
- It is centred on architecture,
- It is iterative and incremental.

a) UP is driven by use cases

The main purpose of a software system is to serve its users; it is therefore important to understand the wants and needs of future users. The term user does not only refer to human users but also to other systems. Use cases are not just a tool for specifying system requirements. They will completely guide the development process through the use of models based on the use of UML language.

- From the use case model, developers will create a series of design and implementation models that will realize the use case.
- Each successive models will then be revised to control the conformity of the use cases
- Lastly, the testers will test the implementation to assure that the components of the implementation model corresponds to the use cases.

b) UP is centred on architecture

From the start of the process, we will have a view of the architecture to be put in place. The architecture of a software system can be described as the different views of the system that needs to be built. The software architecture is equivalent to the most significant static and dynamic aspects of the system. Architecture emerges from business needs, as expressed by users and other stakeholders and as reflected in use cases. It is also influenced by other factors:

- The platform on which the system will have to run;
- The reusable building blocks available for development;
- Deployment considerations, existing systems and non-functional needs (performance, reliability, etc.)

c) UP is Iterative and incremental

Developing a software product for commercialization is a huge undertaking that can take several months. We are not going to develop everything at once. We can cut the work into several parts which are as many small projects. Each of them representing an iteration that results in an increment. An iteration means the succession of steps in the sequence of activities, while an increment corresponds to progress in the different stages of development. The choice of what to implement during an iteration is based on two factors:

- An iteration takes into account a certain number of use cases which together improve the usability of the product at a certain stage of development.
- The iteration treats in priority the major risks.

An increment is often an additive. At each iteration, the developers identify and specify the relevant use cases, create a design guided by the chosen architecture, implement this design in the form of components and verify that these comply with the use cases. As soon as an iteration meets the set objectives development moves to the next iteration. To make the development profitable, it is necessary to select the iterations necessary to achieve the objectives of the project. These iterations must follow one another in a logical order. A successful project will follow a direct flow, established from the beginning by the developers and from which they will only deviate very marginally. Eliminating unforeseen problems is part of the risk reduction objectives.

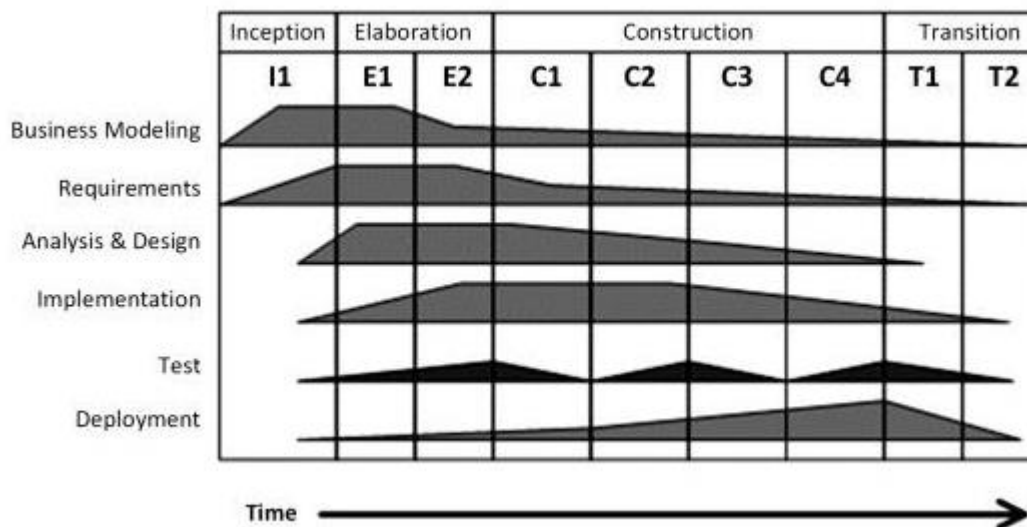
2. UP life cycle

Unified Process is based on the enlargement and refinement of a system through multiple iterations, with cyclic feedback and adaptation. The system is developed incrementally over time, iteration by iteration. The iterations are spread over four phases where each phase consists of one or more iterations.

- **Inception (Creation)**—the first and the shortest phase in the project. It is used to prepare basis for the project, including preparation of business case, establishing project scope and setting boundaries, outlining key requirements, and possible architecture solution together with design tradeoffs, identifying risks, and development of initial project plan—schedule with main milestones and cost estimates. If the inception phase lasts for too long, it is like an indicator stating that the project vision and goals are not clear to the stakeholders. With no clear goals and vision the project most likely is doomed to fail. At this scenario it is better to take a pause at the very beginning of the project to refine the vision and goals. Otherwise it could lead to unnecessary make-overs and schedule delays in further phases.
- **Elaboration**—during this phase the project team is expected to capture a majority of system's requirements (e.g., in the form of use cases), to perform identified risk analysis and make a plan of risk management to reduce or eliminate their impact on final schedule and product, to establish design and architecture (e.g., using basic class diagrams, package diagrams, and deployment diagrams), to create a plan (schedule, cost estimates, and achievable milestones) for the next (construction) phase.
- **Construction**—the longest and largest phase within Unified Process. During this phase, the design of the system is finalized and refined and the system is built using the basis created during elaboration phase. The construction phase is divided into multiple iterations, for each iteration to result in an executable release of the system. The final iteration of construction phase releases fully completed system which is to be deployed during transition phase
- **Transition**—the final project phase which delivers the new system to its end-users. Transition phase includes also data migration from legacy systems and user trainings.

Each phase and its iteration consists of a set of predefined activities. The Unified Process describes work activities as disciplines—a discipline is a set of activities and related artefacts in one subject area (e.g., the activities within requirements analysis). The disciplines described by Unified Process are as follows

- Business modelling—domain object modelling and dynamic modelling of the business processes,
- Requirements—requirements analysis of system under consideration. Includes activities like writing use cases and identifying non-functional requirements,
- Analysis and design—covers aspects of design, including the overall architecture,
- Implementation—programming and building the system (except the deployment),
- Test—involves testing activities such as test planning, development of test scenarios, alpha and beta testing, regression testing, acceptance testing, and
- Deployment—the deployment activities of developed system.



It clearly shows that the relative effort across disciplines changes over time from iteration to iteration, e.g., initial iterations apply greater relative effort on requirements and design while the latter—more on testing and deployment.

3. Example of a software development process that implements the unified process: 2TUP (Two tracks unified process)

In the 2TUP process, development activities are organized according to 5 workflows which describe:

- Capturing needs,
- Analysis,
- Design,
- Implementation
- And the test.

The 2TUP process is a frame of best development practices, it should be used as a guide to carry out a project and not as the ultimate and universal weapon of development.

a) The 2TUP development cycle

The 2TUP offers a Y-shaped development cycle, which separates the technical aspects from the functional aspects. It begins with a preliminary study which essentially consists in identifying the actors who will interact with the system to be built, the messages that the actors and the system exchange. Then to produce the specifications and to model the context. The process revolves around 3 essential phases: a technical branch; a functional branch and a realization branch.

The functional branch

The functional branch capitalizes on the knowledge of the business of the company. This branch includes:

- The capture of functional needs, which produces a model of needs focused on the business of the users. It qualifies as early as possible the risk of producing a system that is unsuitable for users. For its part, the prime contractor consolidates the specifications and checks their consistency and completeness.
- The analysis, which consists in precisely studying the functional specification in order to obtain an idea of what the system will achieve in terms of business. The results of the analysis do not depend on any particular technology.

The technical branch

The technical branch capitalizes on technical know-how and/or technical constraints. The techniques developed for the system are independent of the functions to be performed. This right branch includes:

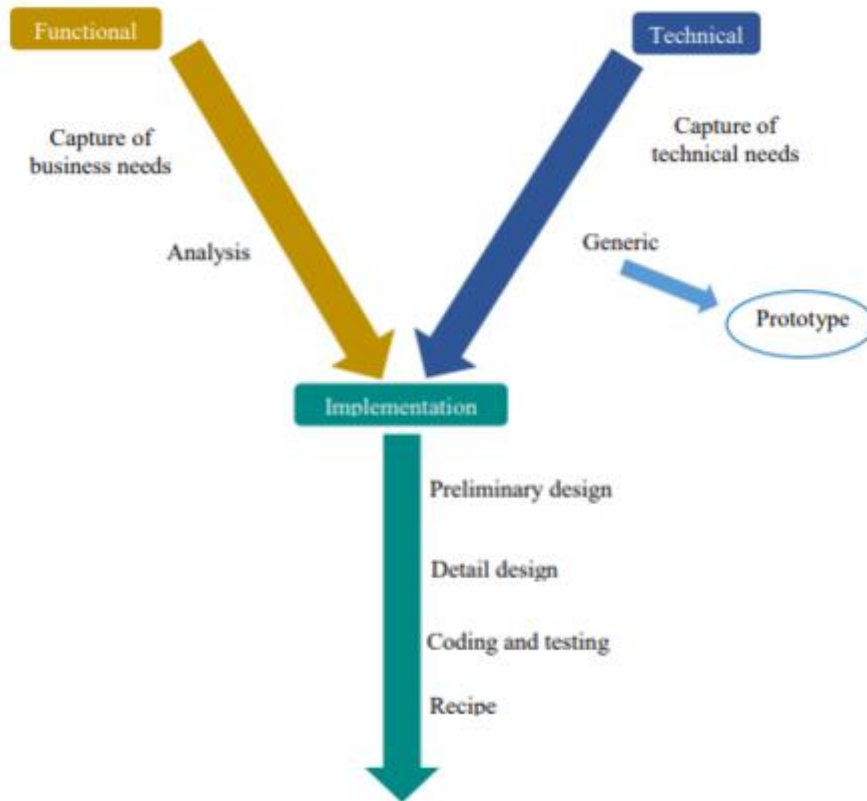
- The capture of technical needs, which lists all the constraints and choices determining the design of the system. The tools and materials selected as well as the consideration of integration constraints with the existing system generally condition the technical architecture prerequisites;
- The generic design, which defines the components necessary for the construction of the technical architecture. This design is completely independent of the functional aspects. Its objective is to standardize and reuse the same mechanisms for an entire system. The technical architecture builds the backbone of the computer system and eliminates most of the technical level risks. The importance of its success is such that it is advisable to make a prototype to ensure its validity.

The realization phase

The implementation phase consists of bringing the two branches together, allowing application design to be carried out and finally the delivery of a solution adapted to the needs. The middle branch includes:

- The preliminary design, which represents a delicate stage, because it integrates the analysis model into the technical architecture so as to draw the map of the components of the system to be developed,

- The detailed design, which studies how to make each component;
- The coding stage, which produces the components and progressively tests the code units produced,
- The acceptance (recipe) stage, which finally consists in validating the functions of the developed system



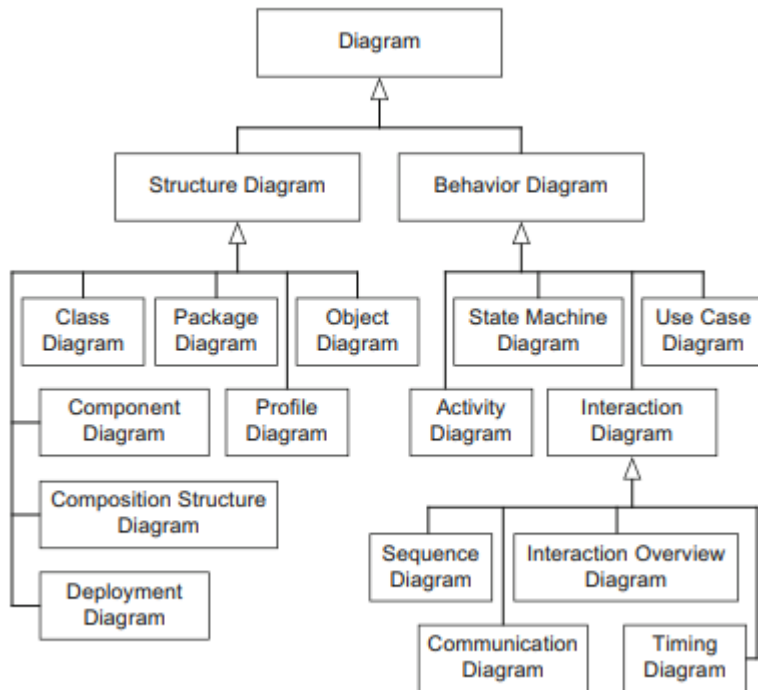
III. The Views

A way of implementing UML is to consider the different views that can be grouped together in order to collaborate and define the system.

- *The use case view*: it's the description of the model seen by the actors of the system. It corresponds to the need expected by each actor (package diagram and use-case diagram).
- *The logical view*: it's the definition of the system seen internally, it explains how the need of the actors can be satisfied. It describes the system in term of classes, objects and communication. It identifies the elements of the domain, the relations and the interaction between these elements (class diagram and object diagram).
- *The component view*: it's highly the different parts that would make up the future system (source files, library, databases, executable etc.). This view includes 2 diagrams (the composite structure diagram and the component diagram)
- *The process view*: it describes the interaction between process, synchronization and communication of parallel activities. It includes the sequence diagram, activity diagram, collaboration diagram, state machine diagram, interaction overview diagram and timing diagram.
- *Deployment view*: This view describes the physical architecture of each element of the system and the distribution of the pass of the software of these elements. It includes the deployment diagram

IV. Diagrams

UML 2.5 has 14 diagrams divided into 2 main groups



This figure shows a taxonomy of the 14 UML diagrams, giving a very rough categorization. As the figure shows, we differentiate between structure diagrams and behaviour diagrams

- **Behavioural or dynamic diagram**
 - *The use-case diagram*
 - *The activity diagram*
 - *The state machine diagram.*

In these categories of behavioural diagram, we have diagrams called

- **Interaction diagrams:**
 - *communication or collaboration diagram (since UML 2.0)*
 - *the sequence diagrams*
 - *the timing diagram (since UML 2.0)*
 - *the interaction overview diagram (since UML 2.0)*

- **Structural or static diagrams:**

- Class diagram
- Package diagram
- Composition structure diagram
- Object diagram
- Deployment diagram
- Component diagram
- Profile diagram (since UML 2.2)

A. *Behavioural or dynamic Diagrams*

Behavioural diagram does not deal with statics structures. It rather shows the existing process flow as expected from the use of the program or software. Behavioural diagrams describes the system in term of interactions on 2 plans:

- **The system seen as a black box:** it shows who interact with the system and for what purpose (Use case diagram). How does the system interact with its environment (sequence and activity diagrams).
- **System seen as a white box:** It shows how the objects of the system interact (sequence and or communication diagram). It describes the evolution of the system over time, how the state of the object evolve (state machine diagram).

1. Use case diagram

Use case diagrams describes the high-level functions and scope of a system. These diagrams also identify the interaction between the system and its actors. The use cases and actors in use case diagrams describe what the system does and how the actors use it but not how the system operate internally. Thus, use case diagram allow to identify the demand or expectation toward the system.

Specifically, we can employ a use case diagram to answer the following questions:

1. What is being described? (The system.)
2. Who interacts with the system? (The actors.)
3. What can the actors do? (The use cases.)

- i. **An actor** is a model element that interacts with a system, it can be an abstract person (e.g.: customer etc.) or another external system.

The role of actor can be attributed to a service, a society and API etc.

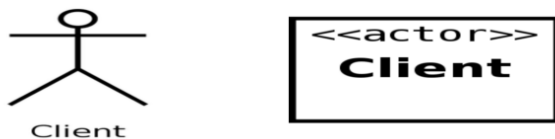
A person can be represented by several actors depending on the roles they play (example: the admin and director roles can be assigned to the same person).

To identify the actors of the system, it is necessary to focus on the roles played by the entities out of the scope.

NB: In UML there is no notion of internal and external actor, by definition an actor is out of the scope of study whether he belong or not to the company.

There are 2 main categories of actors

- **The principal/primary/main actors:** These are the people who use the main function of the system.
- **The secondary actors:** These are entities that provide services to the system.

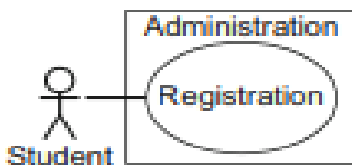


UML offers the use case diagram to enable you to define the required Use case diagram.

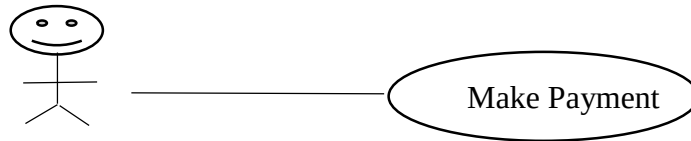
- ii. **The Use case:** A use case corresponds to the objective of a system motivated by the need of the actors. The set of use cases describe the objective or goal of a system.



Registration Student Administration mentions that a system must fulfil. This diagram describes which users use which functionalities of the system but does not address specific details of the implementation. The units of functionality that the system provides for its users are called **use cases**. In a university administration system, for example, the functionality *Registration* would be a use case used by students. It is called **the association**

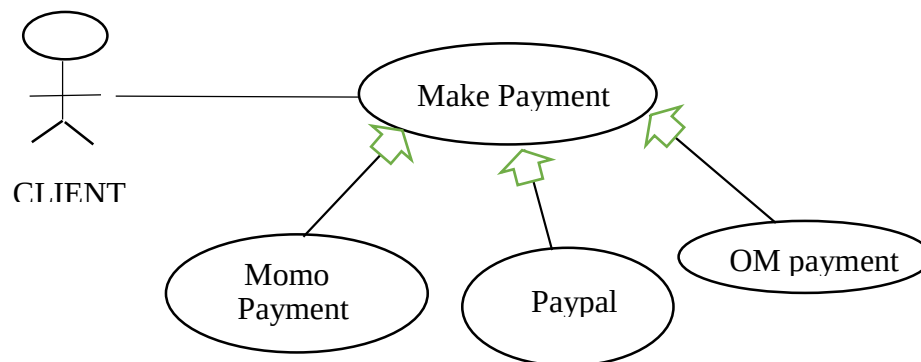


NB: A use case is a verb in the infinitive form + the nominal group.

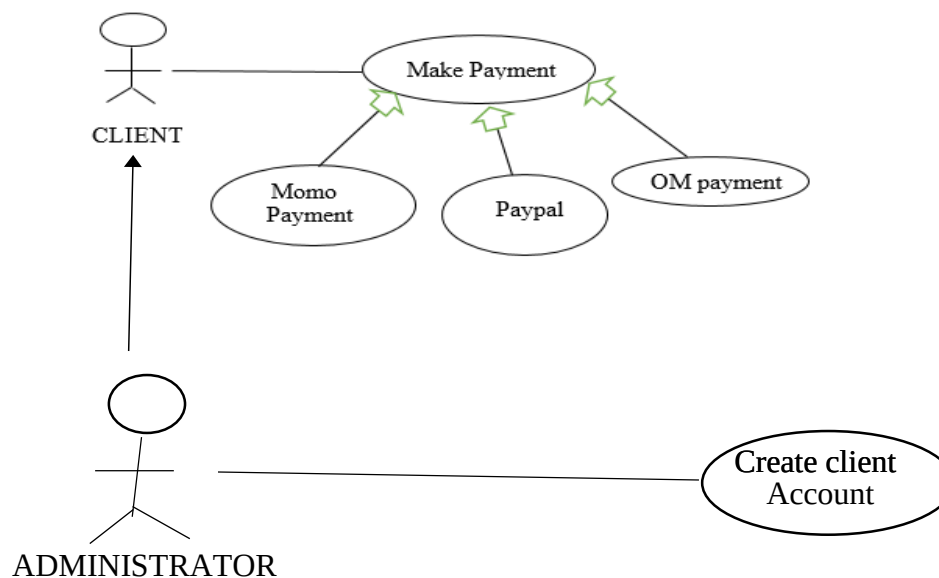


There are 3 types of relationships between different use cases

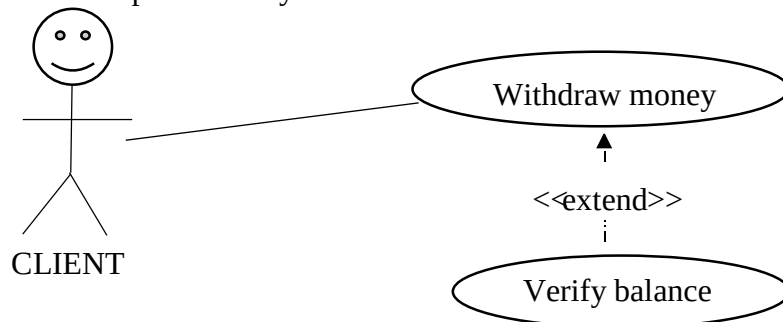
- iii. **The Generalisation / specialisation relationship:** In a general relation between 2 use cases, the child use case is a specialisation of the parent use case. This relationship is useful for showing that one use case is a special type of another and the more specialise use case differs somehow from the original. Thus, all the step of the original use case must be executed



NB: An actor can also participate in generalisation relationship with other actor, the child actor would then be able to communicate with the same use cases as the parent actors by **Inheritance**.

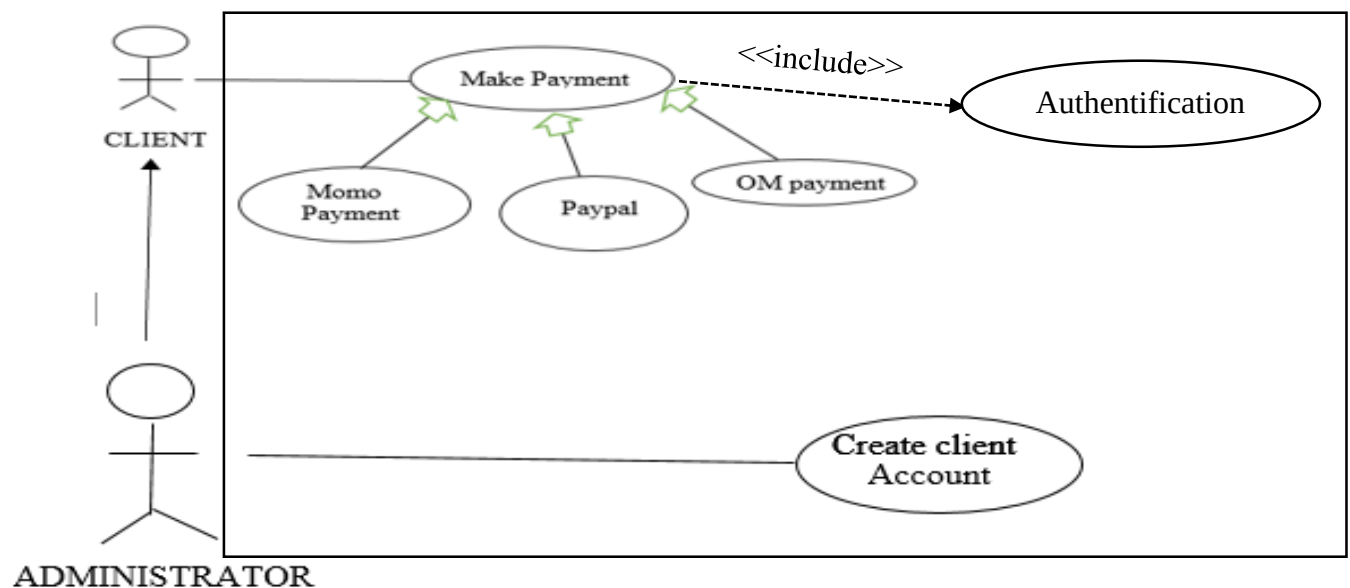


- iv. **Extension relationship:** it indicates that the source use case adds its behaviour to the destination use case. The use case is optional and comes after the based (main) use case, it is represented by a dash arrow in the direction of the based use case with the notation **extend**.



NB: A use case can have several extension points.

- v. **The Inclusion Relationship:** The use case is mandatory and part of the based use case. It is represented by a dashed arrow in the direction of the included use case with the notation **include**.



vi. **The System**: it shows the formalism in which a use case is related.

• Human/non-human • Active/passive • Primary/ secondary

An actor interacts with the system by using the system as an active actor, meaning that the actor initiates the execution of use cases;

If the interaction involves the actor being used by the system, meaning that the actor is a passive actor providing functionality for the execution of use cases.

The secondary actor does not necessarily have to be passive. Both the Professor and the Student are actively involved in the execution of the use case

a) Textual description of use cases

The description of a use case is done by scenarios that describe the logical sequence of action that constitute this use case. This description specifies what the actor does and what the system does. It clarifies the progress of the functionality by describing the chronology of the action which will have to be carry out. It has 3 main parts

Part 1: Identification:

- Title: name of the use case
- Summary: Description of the use case.
- Actor: Description of the main and the secondary actors.
- Date: the creation and the update date
- Stakeholder: Names of the stakeholders
- Version: The version number

Part 2: Description of scenarios

- Preconditions: It is the state of the system before the use case can be triggered
- Triggers: This is the event that causes the use case to be initiated.
- Scenarios: There are several types of scenarios
 - ✦ **The nominal scenario** which corresponds to a normal development of a use case.
 - ✦ **The alternative scenario** which are variant of the nominal scenario. This is the class for steps link to conditions
 - ✦ **The Exception scenarios** we talk of exception scenario when a step in the process could be disrupted due to an abnormal event

- ✦ **Post condition:** They describes the state of the system after the end of each scenario

Part 3: Non-functional requirement (reliability performance)

NB: A textual description is use to clearly narrate the expected system responses and outcomes, it shows what the system will use for the development and also the importance for testing them.

Example: Textual description of the use case authenticate

Title: Textual description of a use case authenticate

Summary: This use case allows users to have access to the platform

Actors: All users

Date: 16/12/2023

Manager: Mr Messio

Version: 1.0

Precondition: The user has an account on the platform.

Trigger: The user clicks on the authenticate buttons.

Scenarios:

-The Nominal scenario:

1. The user enters the URL of the platform
2. The system displays the login form
3. The user enters his login and password
4. The system verifies the conformity of the field
5. The system sends the data to the database in form of query
6. The database return the result of the query
7. The system displays a success message to the user

-The alternative scenario:

4.a. In step 4 of the nominal scenario, the user enters wrong credential

4.b. The system displays an error message then return to step 2 of the nominal scenario.

6.a at this level if there is no correspondence in the database the system is supposed to display an error message and go back to step 2 of the nominal scenario

-Exception scenario

3.a The user enters incorrect or missing identifiers more than 3 times

3.b The system goes into stand by for 15 second then return to step 2 of the nominal scenario

- Post condition of success:

The user has access to his work space

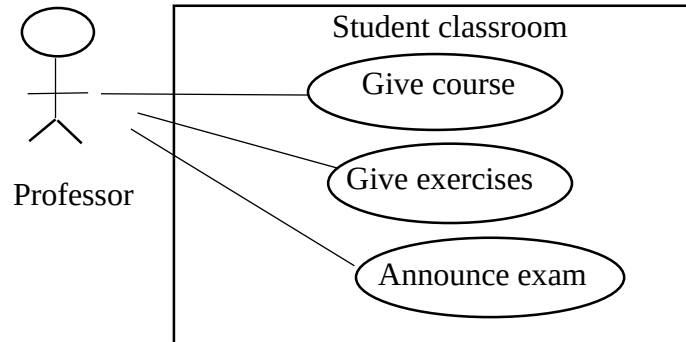
- **Post condition of failure:**

The user does not have access to the platform

- **The none functional requirement**

The Entry of the password must not be visible on the screen

For example this Figure bellow shows the Student Administration system, which offers three use cases: (1) Give course, (2) Give exercises, and (3) Announce exam. These use cases may be triggered by the actor Professor



Practical case 1

On an e-commerce website, a user can select products possibly after reading the data sheet of each product. He can then view his basket. If he decides to validate his basket, he must give his contact details and he can either choose to pay online or pay at the reception. Draw the corresponding use case diagram of this system.

Practical case 2

At UBA, the management process of obtaining visa bank cards is as follows:

A client wishing to obtain a visa card must request it from an account manager. The card is not granted if the applicant is not a customer of the bank. Every day, the credit card office receives requests, identifies them, processes them, classifies them and transmits to the card management service for verification of the conformity of information sent by customers. At the end of the month, he is responsible for ensuring that the list of applicants are sent to the international bank card centre (CICB) for establishment. As soon as the bank has received all the cards from the centre, the card office sends the customers via its manager a notice of availability and notice of deduction of annual fee. The customer then comes to collect his card and signs an acknowledgement of receipt. If after two months the card has not been withdrawn, it's cancelled.

2. Activity Diagram

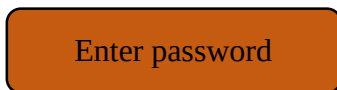
An activity diagram provides a behaviour view of the system by describing the sequence of action in a process or a particular activity.

The activity diagram makes it possible to emphasis the processing and to graphically formalise the sequence of action carried out in the use case.

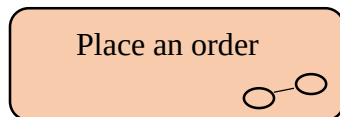
a) Elements of an activity diagram

- i. **Action:** it is an elementary and instantaneous operation. It is a discrete step from which a behaviour is build.

Formalism



NB: It should be noted that there are several types of action. Also, if an action of a use case correspond to the called of an internal use case, it is represented by an action containing a special site that is 2 circle connected by a line.



- ii. **An activity:** It define a behaviour describe by an organise series of units whose simple elements are the action. An activity is a complex operation that can be broken down into action.

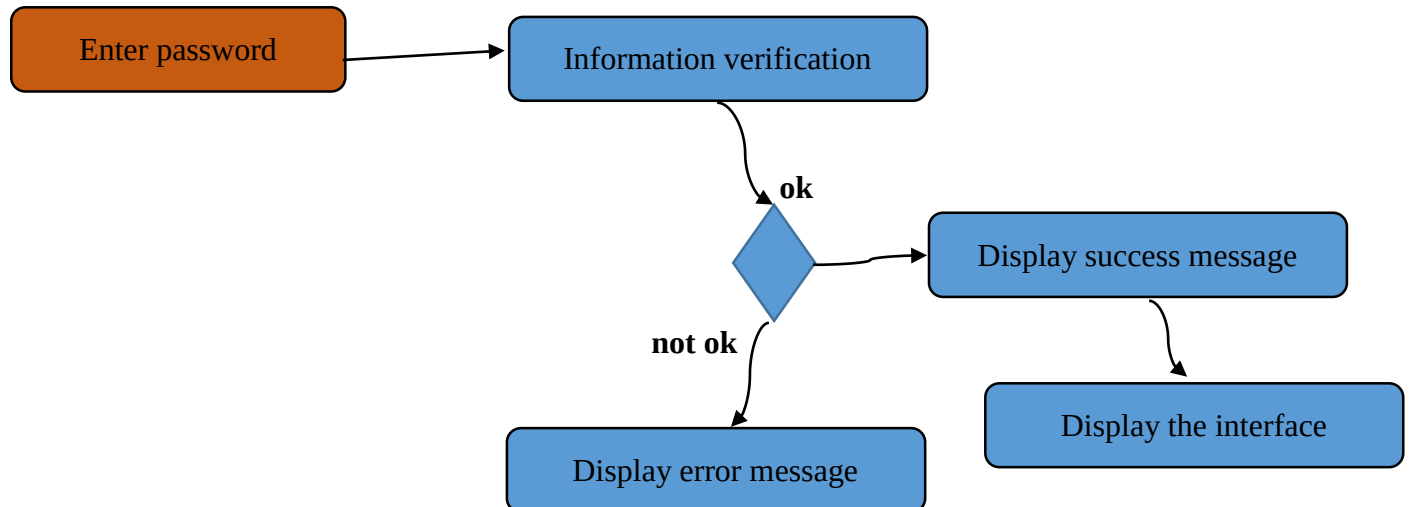
- iii. **A transition:** It represent the package from one action to another. It is represented by an arrow.

Formalism



- iv. **Decision node:** It represent a decision and always have at least 2 paths branching out with condition text to allow user to view options.

Formalism



- v. **The initial node:** It represent the beginning of a process or workflow in an activity diagram.

Formalism



- vi. **Final node:** It marks the end state of an activity and represent the completion of all flow of the process. The exits 2 types of final node: **End point and flow final node**

When one of the arcs of an activity is activated, execution of the development.

End of ALL execution paths of an activity



End of ONE execution path of an activity

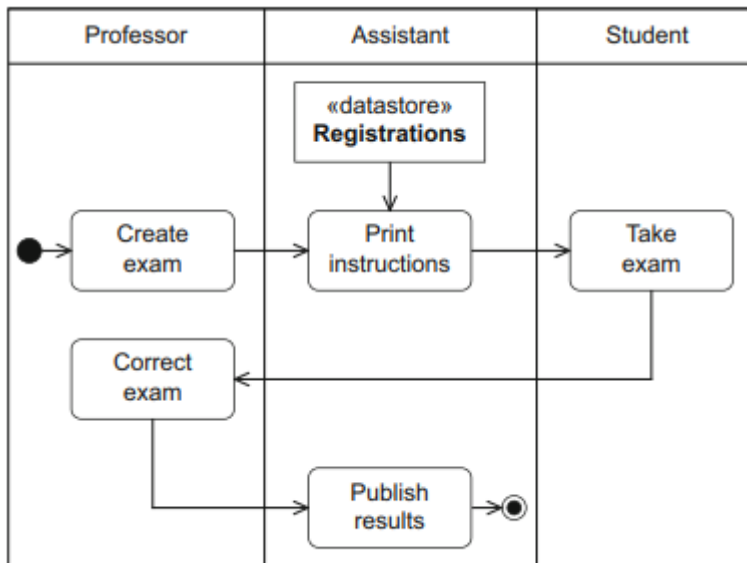


- vii. **Partitions:** are used to organize the activity nodes arranged in an activity diagram through groupings.

Graphically, the scores are represented by solid lines. They can take the form of a table.

In addition, the activity nodes must belong to a single partition, and the transitions can pass across partition boundaries.

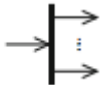
Example of a partition:



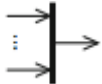
- viii. **Bifurcation node:** or disconnect node is a control node that separates a stream
- ix. or several competing streams. It therefore has an incoming arc and several outgoing arcs. It is represented by a thick solid line



- If an execution path splits into multiple simultaneously active execution paths later on, you can realize this using a **parallelization node**. A parallelization node is depicted ... as a black bar with one incoming edge and multiple outgoing edges

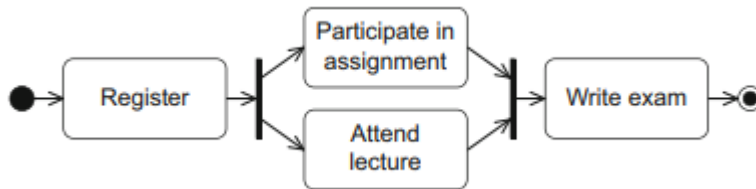


- You can merge concurrent sub paths using a **synchronization node**. This node is the counterpart to the parallelization node. It is depicted as a black bar with multiple incoming edges but only one outgoing edge.



Example of the use of parallelization and synchronization nodes

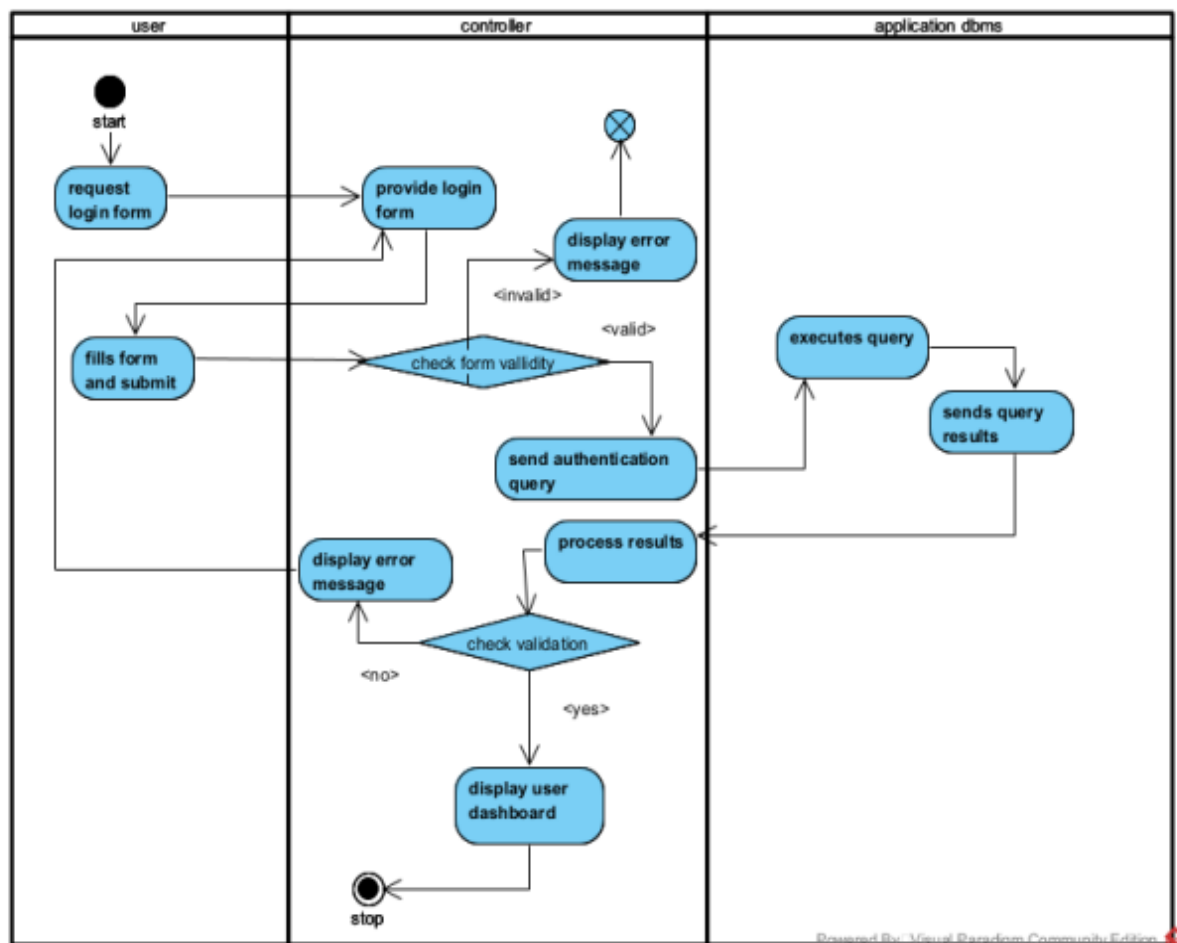
MR. MESSIO, Computer Engineer, Master Degree in IS's Management / Lecturer in AICS



Once a student has registered for a course, the student attends the lecture and participates in the assignment simultaneously. The student can only write the exam when both the lecture and the assignment have been completed.

Example:

Activity diagram for authentication



3. Sequence diagram

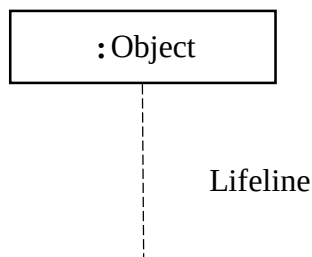
A sequence diagram is an UML diagram that illustrate the sequence of messages between object in an interaction. It consists of a group of object that are represented in lifelines and the message they exchange over time during the interaction. Interaction is arranged from top to bottom following their order of occurrence. The goal of a sequence diagram is to:

- Represent the detail of the use case
- Model the logical flow of a complex procedure function or operation.
- Model the high-level interaction between acting objects in a system
- Either model generic interaction (showing all possible path through the interaction) or specific instances of an interaction (showing just 1 path through the interaction).

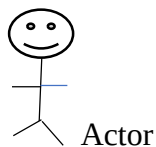
a) Component of a sequence diagram

- i. **The lifeline:** It is a dotted vertical line that shows the sequential event affecting an object during the process.

- ii. **Object:** It is the instance of a class



- iii. **Actor:** An actor communicates with object it is represented as follows



- iv. **Activation:** it represents the time require for an object or an actor to complete a task it indicates when the object perform an action. The longer the task requires, the longer the activation box is.



- v. **Message:** A message is model by a horizontal arrow between activation. It indicates the communication between object. There are several types of messages in a sequence diagram.
- **The synchronous message:** The reception of a synchronous message must cause the recipient to launch one of its operation or method. The sender of the message remain block during the entire execution of the method and therefore wait for the end of the method before been able to launch a new message. It is represented as follows

Synchronous message



- **The asynchronous:** they do not require a respond from the recipient before the sender continuous. They are represented as

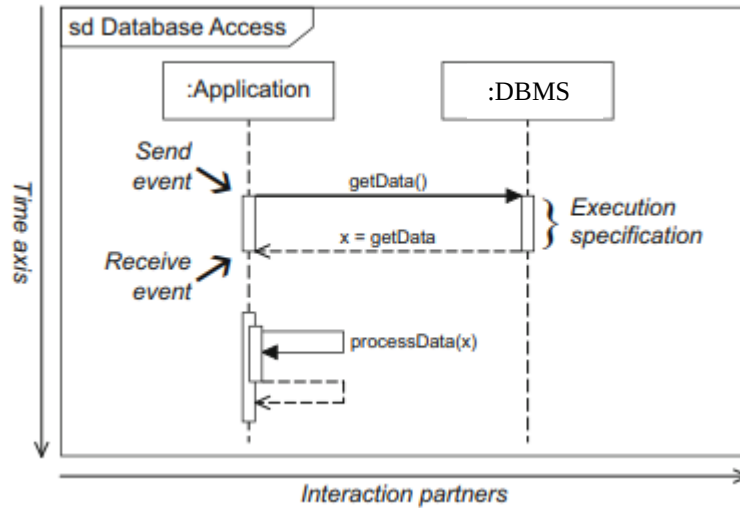
Asynchronous message



- **The reply or respond message:** They represent the response to a message

Response message





b) Structure of a sequence diagram

1- The combine interaction fragment: They represent an articulation of operation. It is defined by an operands or operator. The operator conditions the meaning of the fragment. They are 12 operators defined in UML 2.0 notation. Combine fragment make it possible to describe sequence diagram in a compact way. A combine fragment is represented by a rectangle whose upper left corner contain a pentagon. In the pentagon there is a type of combination called the **interaction operator**. **Operators for combined fragments**

	Operator	Purpose
Branches and loops	alt	Alternative interaction
	opt	Optional interaction
	loop	Iterative interaction
	break	Exception interaction
Concurrency and order	seq	Weak order
	strict	Strict order
	par	Concurrent interaction
	critical	Atomic interaction

6.1. Branches and loops

the following is a list of interaction operator by function

- **An Alternative interaction operator** has at least two operands. Each operand represents an alternative path in the execution, which corresponds approximately to multiple cases in programming languages for example, the switch statement in Java. Guards are used to select the path to be executed.

- **Choice and loop operator:** Alternative, options, break and loop. The **alt operator** is an alternative. It represents 2 possible behaviours. It is an equivalent of the **IF...THEN...IF NOT**. Therefore, only one of the 2 branches will be performed in a given scenario. The execution condition of one of the 2 branches (the equivalent of the IF) can be explicit or implicit. The use of **THEN** allow to indicate that the branch is executed if the condition of the **alt** is false
- **The OPT operator:** The OPT operator denoted an optional combine fragment. It represents a behaviour that may or may not happen. An optional fragment is equivalent to an out fragment

Example of an **alt** and an **opt** fragment

When a student wants to register for an exam, the following cases can occur: (1) there are still places available and the student can register. (2) There is a place available on the waiting list. Then the student has to decide whether to go on the waiting list. (3) If there is no place available for the exam or on the waiting list for the exam, the student receives an error message and is not registered for the course. *Figure 1*

Example of a **break** and **loop** fragment

You can use the loop fragment to express that a sequence is to be executed repeatedly. This combined fragment has exactly one operand. The keyword loop is followed by an optional specification of the number of iterations of the loop

The break fragment has the same structure as an opt operator, that is, it consists of a single operand plus a guard. If the guard is true, the interactions within this operand are executed, the remaining operations of the surrounding fragment are omitted, and the interaction continues in the next higher level fragment. *Figure 2*

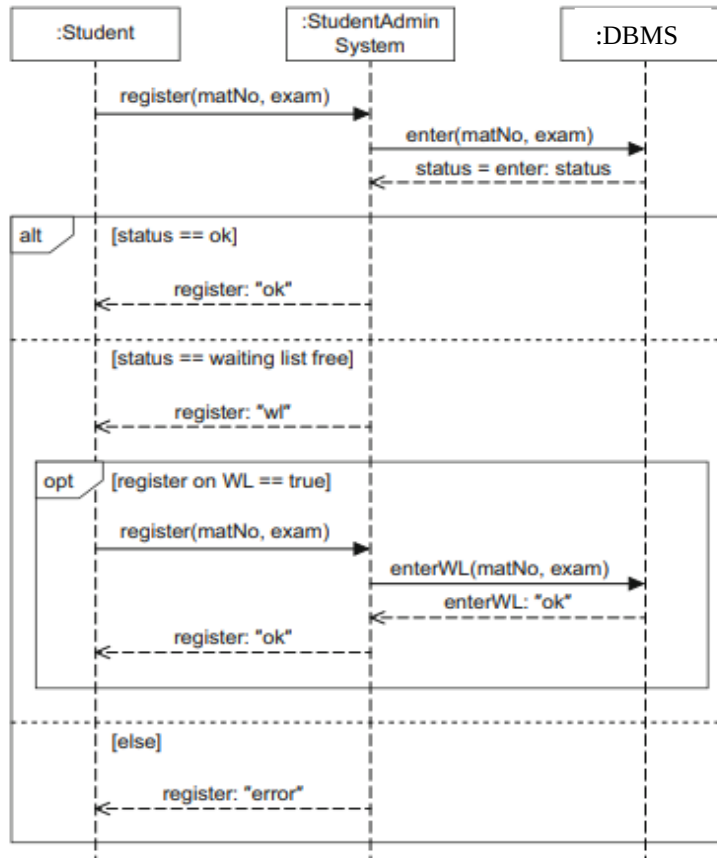


Figure 1

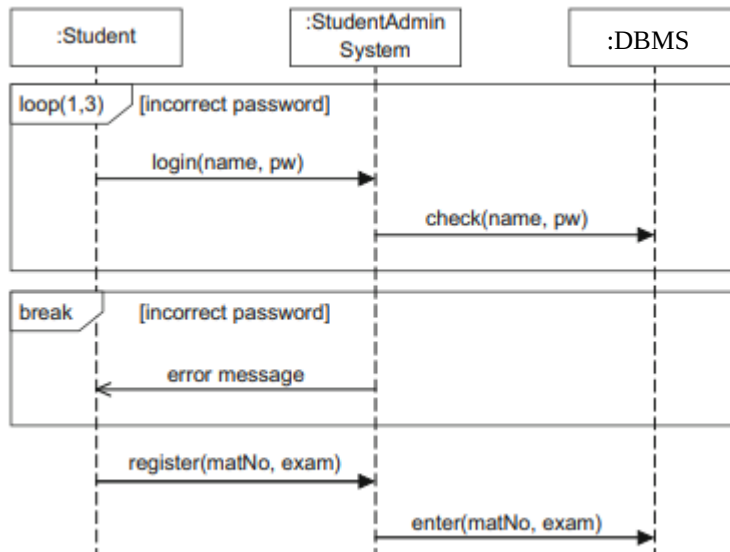
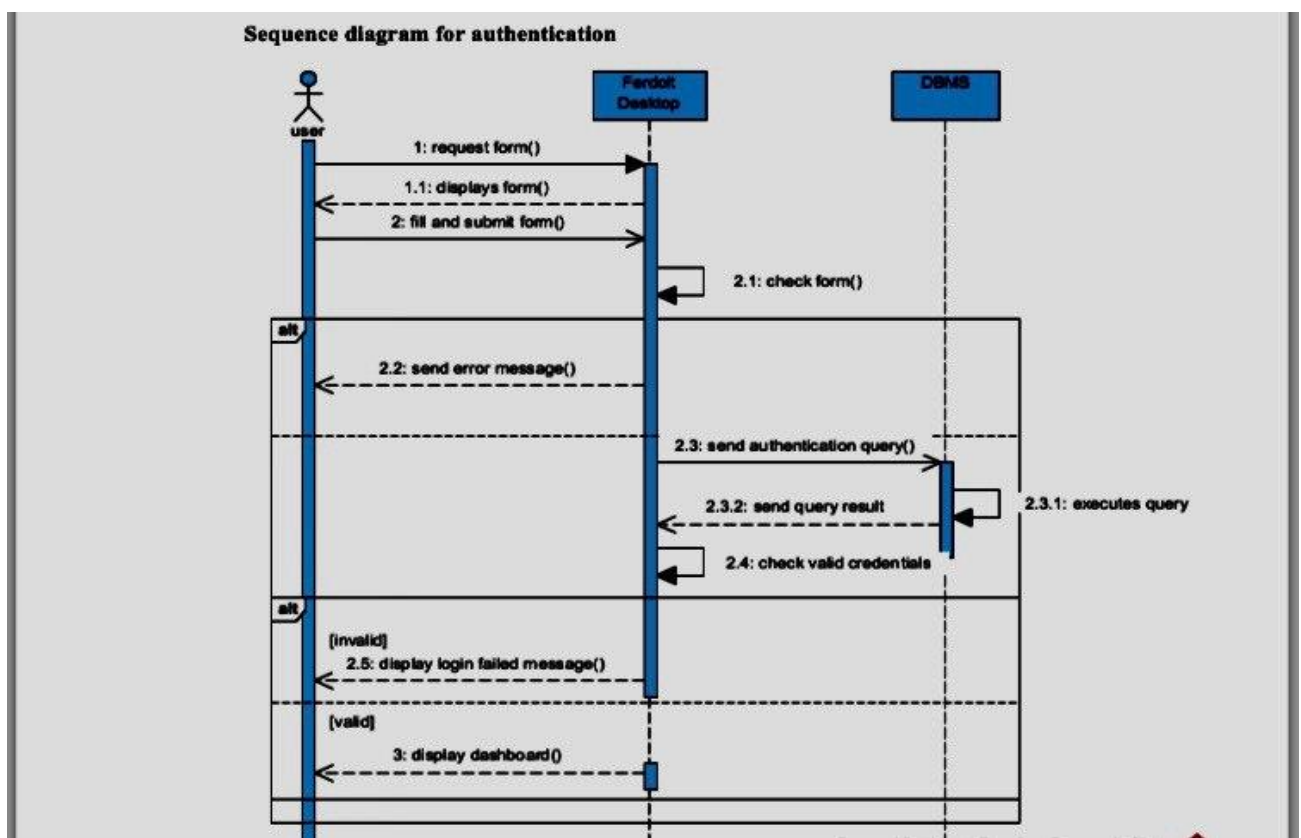
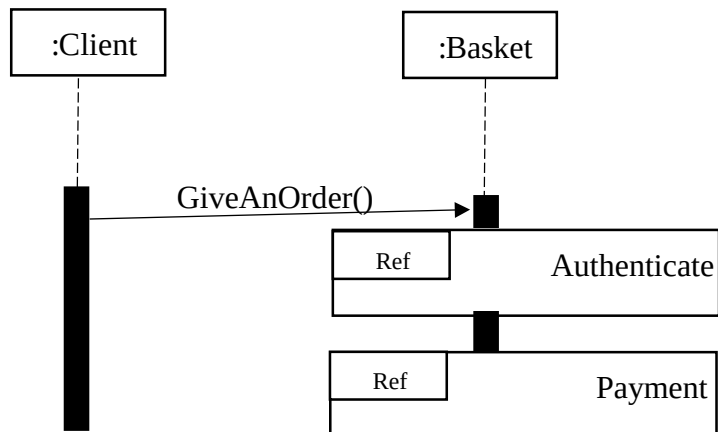


Figure 2

- **The reuse sequence operator:** reference (ref) which indicate the reuse of a sequence diagram.

Example:



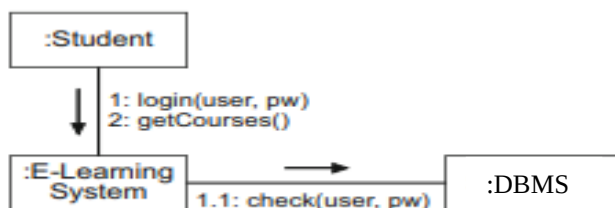
4. Collaboration or Communication Diagram

Similar to a sequence diagram, communication diagrams models message forwarding using lifelines by numbering the sequences. These numbers represent the order in which messages are sent. On communication diagrams, objects are presented with connectors association between them. Messages are added to these associations, and the arrows show the direction of the message flow.

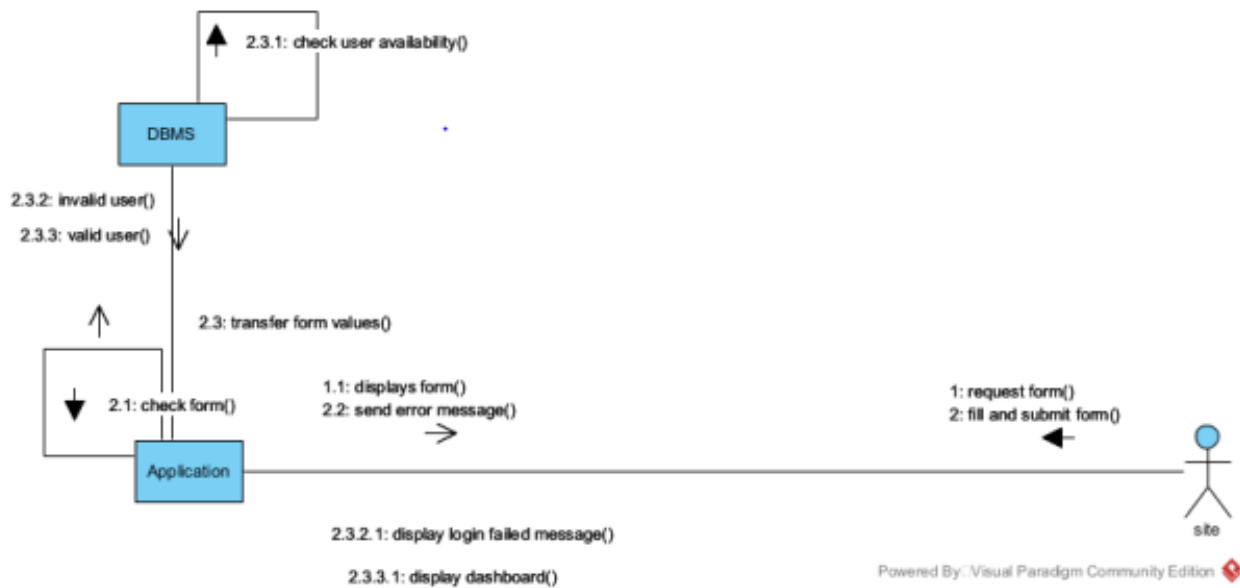
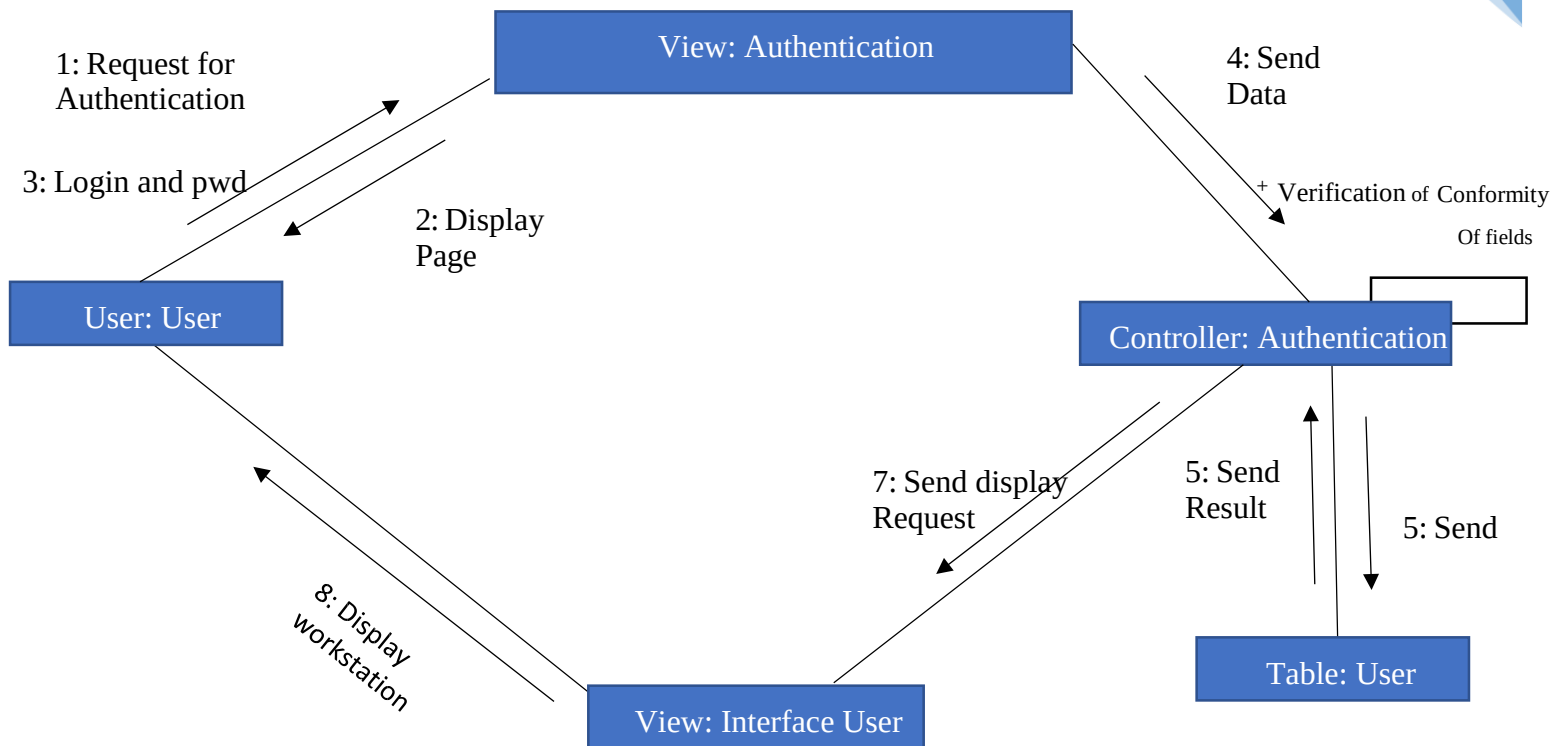
- **The lifeline:** is represented by a rectangle containing the name and type of the proceeding.
- **Connectors:** The relationships between lifelines are called **connectors** and are represented by a solid line connecting two lifelines.
- **Messages:** refer to a particular communication between lifelines, and are usually ordered in ascending order of number.

The communication diagram is a structured diagram that models the Communication diagram relationships between communication partners. It therefore shows directly who communicates with whom. The relationships are the result of the exchange of messages. Here, time is not a separate dimension. The order in which the messages are exchanged is expressed using decimal classification (sequential numbering) for the messages.

The figure below shows the log-in process as a communication diagram. Again, the interaction partners are student, e-learning system, and dbms. The diagram shows that the student communicates with the e-learning system twice: once using login (user, PW) and once using getCourses(). It also shows that the e-learning system communicates with the database using check (user, pw).



Example of an Authentication communication diagram with MVC model



Example of an Authentication communication diagram within a system

MR. MESSIO, Computer Engineer, Master Degree in IS's Management / Lecturer in AICS

5. State Machine Diagram

UML state machine diagrams describe the internal behaviour of an object. They present the possible sequences of states and actions that a class instance can process during its lifetime. A state represents a period in the life of an object during which it waits for an event or performs an activity. A transition status diagram basically makes it possible to represent and describe the state changes of an object related to an event in a system.

To build a state machine diagram, the following questions can be asked

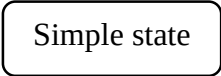
- What happens when an object is in a specific state?
- Which event must occur for its behaviour to be modified?
- What are the triggering events?
- What properties must the object have to be able to change its state?

MODELISATION:

- i. **State:** An object can go through a series of states during its lifetime. A state represents a period in the life of an object during which it waits for an event or performs an activity by satisfying a number of conditions.

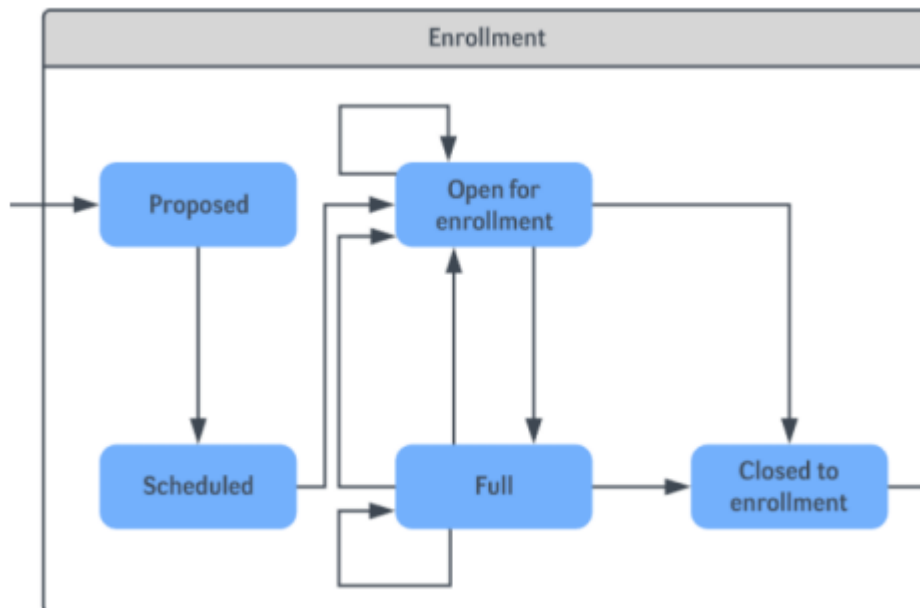
There are several states:

- Simple state: This is an elementary state that does not contain an internal transition or of sub states, it is represented by a rectangle with rounded corners

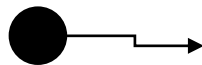


Simple state

- Composite state: it enclose different states



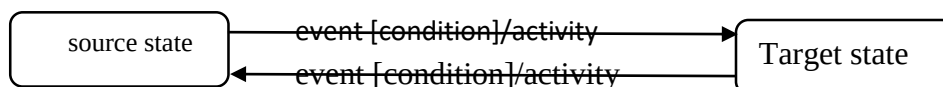
- ii. **Initial node**: It defines the initial state or beginning of a system and is represented by a black filled circle.



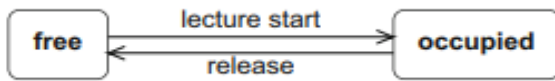
- iii. **Final node**: It represents the final state and end of a system. It is represented by a full circle within a circle.



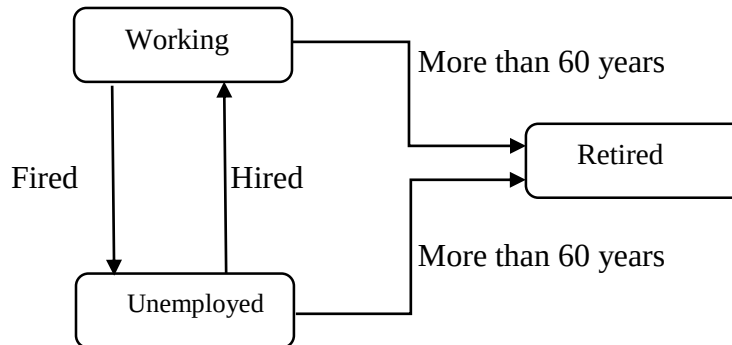
- iv. **Transition**: It represents the transition from one state to another: it is represented by a solid arrow with the name of the event (that triggers the transition) written above or next to it. It can also have a condition or list of actions.



As a simple example consider a lecture hall that can be in one of two states: free or occupied. When a lecture starts in the lecture hall, the state of the lecture hall changes from free to occupied. Once the respective event in the lecture hall has finished and the hall has been released again, its state reverts to free.



Example: State machine of an engineer



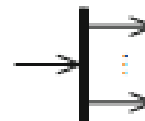
- v. **Decision node:** it is a diamond shape that represent the decision to be made on the bases of an evaluated guard.

You can use it to model alternative transitions. It has exactly one incoming edge and at least two outgoing edges. At the incoming edge, you model the event that triggers the transition; at the outgoing edges, you specify the guards for the alternative paths for the state transition.

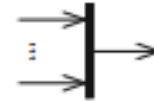
Decision node



- vi. **The parallelization node** is represented with a black bar. It has exactly one incoming edge and at least two outgoing edges and is used to split the flow into multiple concurrent transitions. No events or guards may be specified at the outgoing edges of a parallelization node in a state machine diagram



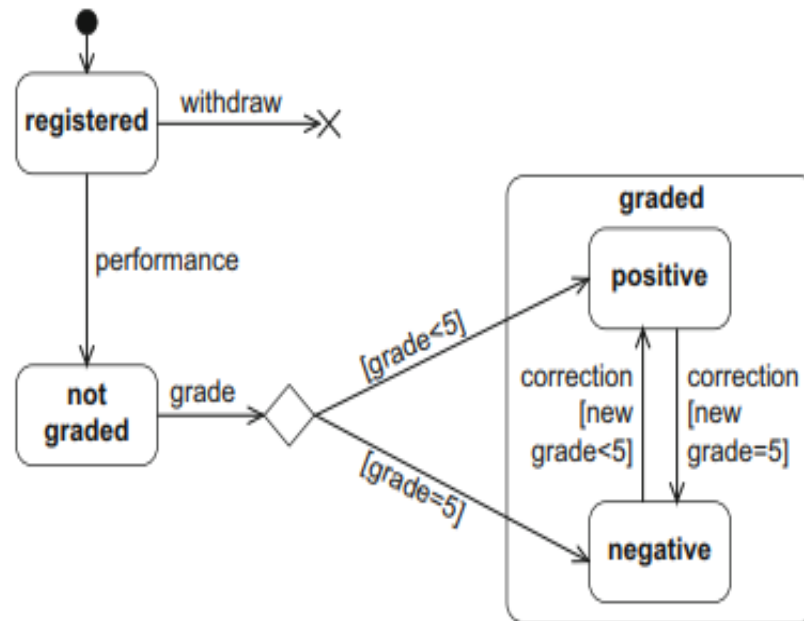
- vii. **The synchronization node** is also represented with a black bar. It has Synchronization node ... at least two incoming edges and exactly one outgoing edge and is used to merge multiple concurrent flows. No events or guards may be specified at the incoming edges of a synchronization node.



- viii. **The terminate node** is represented in the diagram with a large X. If a terminate node is reached in a flow, the state machine terminates and the modelled object ceases to exist.



Example: States of a student's course participation



NB: A state machine diagram can have multiple end states.

6. Timing Diagram

The timing diagram explicitly shows the state changes of the interaction partners that can occur due to time events or as a result of the exchange of messages. For example, a person is in the state logged in as soon as the message is received from the university administration system that the password sent is valid. It must be the representation of the state machine and the sequence diagram.

7. Interaction Overview Diagram

The interaction overview diagram models the connection between different interaction processes by setting individual interaction diagrams (i.e., sequence diagram, communication diagram, timing diagram, and other interaction overview diagrams) in a time-based and causal sequence. It also specifies conditions under which interaction

processes are permitted to take place. To model the control flow, concepts from the activity diagram are used. For example, a user of the university administration system must first log in (which already represents a separate interaction with the system) before being allowed to use further functionalities.

B. Structural or Static Diagrams

They describe the structure of the system in terms of system components: objects, classes, packages, components and the relations between these components (specialisation, association, dependency, etc.).

1. Class diagram

The class diagram expresses the static structure of the system in terms of classes and relations between these classes. The purpose of the class diagram is to model the features of the information system. The class diagram is used to represent all the information that is managed by the domain by describing what the attributes and behaviours, which it has rather than detailing methods to achieve operations. This information is structured, i.e. it has been grouped into classes. The diagram highlights possible relationships between these classes. The diagram of classes has 6 concepts: class, attribute, identifier, relation, operation, generalization / specialization.

a) Modelling element of a class diagram

i. Class

It is an abstract description of a set of objects of an application domain. It defines the structure, the behaviour and the relationship of these objects.

<i>Class name</i>
<i>Attribut1: type</i> <i>Attribut2: type</i>
<i>Operation1()</i> <i>Operation2()</i>

ii. Attribute

An attribute is an elementary piece of information represented by its name and its format that defines a class.

UML defines 3 levels of visibilities for attributes

- **Public (+):** It means that the element of a class is visible to all clients of the class.

- **Protected (#):** the element is visible to subclasses of a class
- **Private (-):** the element is only visible to object of the class in which it is declared.

<i>Class name</i>
+ <i>public</i> <i>Attribut1: type</i>
<i>protected</i> <i>Attribut2: type</i>
- <i>private</i> <i>Attribut3 : type</i>

iii. **Identifier**

An identifier is a particular attribute which makes it possible to uniquely identify each object or instance of a class.

iv. **Operation**

It represent an element of object behaviour defined in a class. An operation is functionally provided by a class. The description of the operation can specify the input and the output parameters as well as the elementary action to be performed.

v. **The relation**

There are several types of relationship between classes: association, generalisation/specialization and dependency.

vi. **An association**

It's a relationship between 2 or more classes. An association thus indicate that there can be a link between instances of associated instances. An association is a relationship between two classes (binary association) or more (n-ary association), which describes the structural connections between their instances.

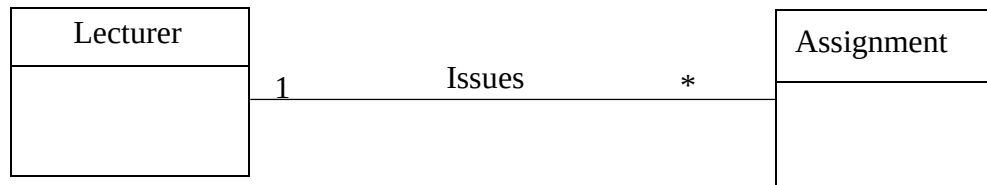
- A binary association is represented by a solid line between classes associated with. It can be adorned with a name, possibly with a clarification of the Reading direction (► or ◄). When the two ends of the association point towards the same class, the association is said to be **reflexive**.
- An n-ary association link more than 2 classes. It is represented by a large diamond with a path leading to each participating classes. If the name of the association exist it is written either in the diamond or beside the diamond.

In an association, we find the notion of **multiplicity** that defines the number of instances of the association for an instance of the class. Multiplicity is defined by an integer or a range of values. The multiplicity is noted on the role (it is noted on the reverse side of the MERISE notation).

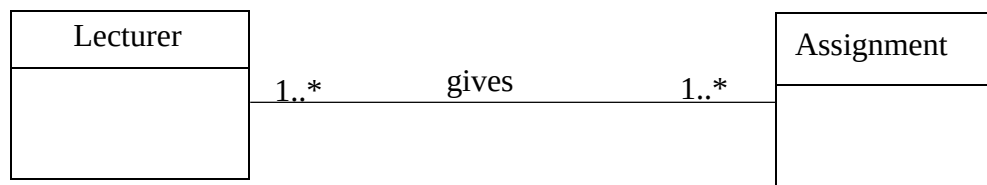
In the same way that multiplicities of attributes and parameters are specified, multiplicities of associations are given as an interval in the form minimum...maximum. They specify the number of objects that may be associated with exactly one object of the opposite side. The values that the minimum and maximum may adopt are natural numbers and an asterisk *, which expresses that there is no restriction. If minimum and maximum are identical, one value and the dots can be omitted. Again, 0...* means the same as *.

Multiplicity	Definition
1	One and only one
0..1	Zero or one
N Or *	Natural numbers
0...*	Zero to many
1...*	One to many

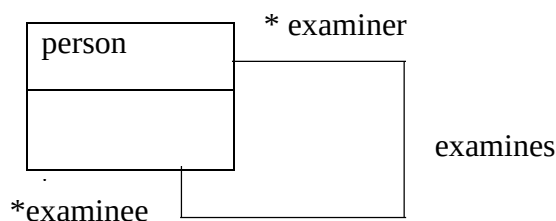
Example of binary association:



This example shows that a lecturer may issue no, one, or multiple assignments and that an assignment is issued by exactly one lecturer. No assignment may exist without an association to a lecturer.



This figure shows that a lecturer gives at least one lecture and a lecture is given by at least one lecturer



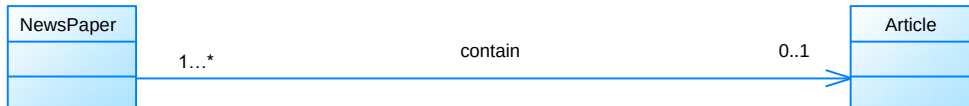
The associations are classified into 3 categories according to the multiplicities they have.

1. **The one-to-one multiplicity**



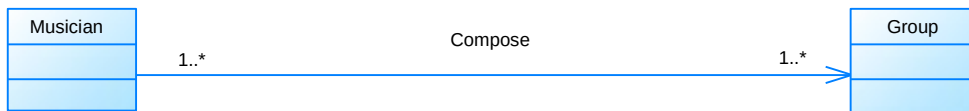
A driver can drive one and only one or zero car while a car may be driven by only 1 driver.

2. **The one-to-many multiplicity**



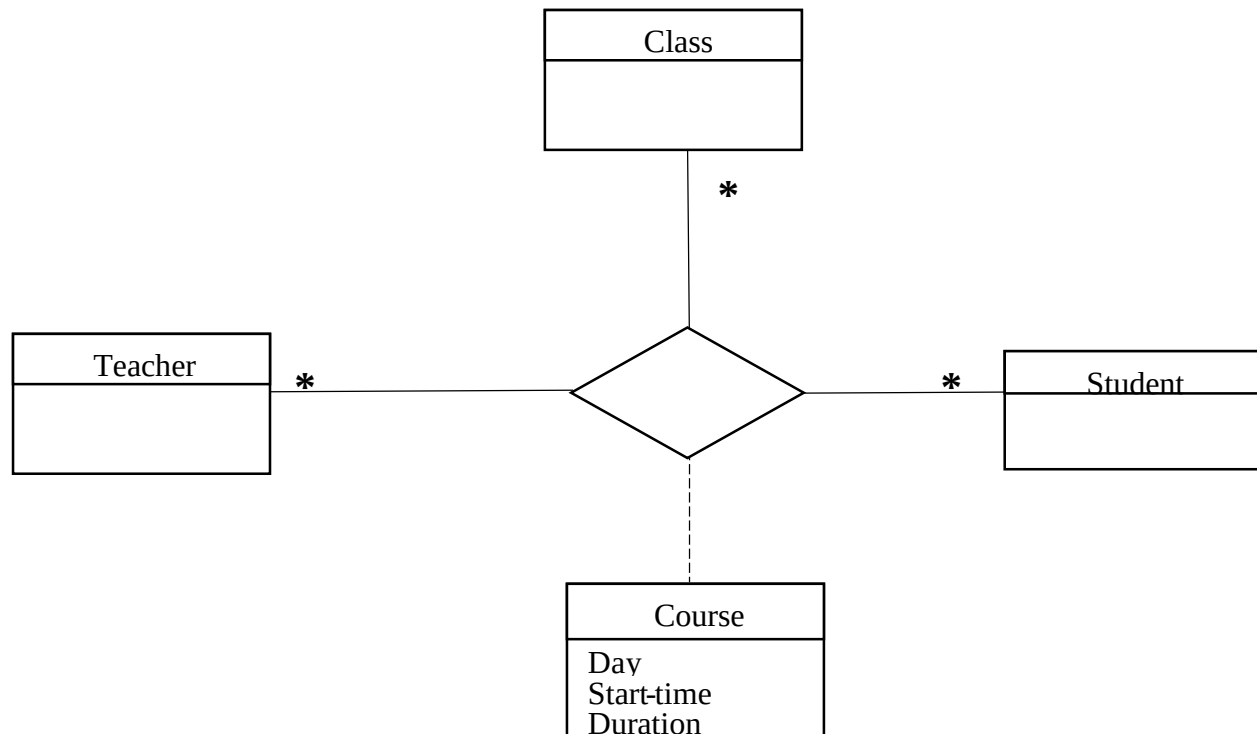
A newspaper contain zero or one Article while an Article may be contained in 1 or many Newspaper.

3. **The many-to-many multiplicity**

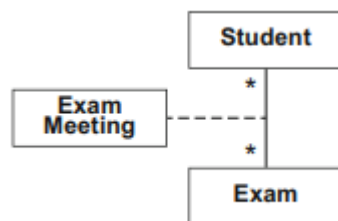


A musician can be found in one or many groups while a group may be composed of one or many Musicians.

Example of an n-ary association



➤ An association class: it's a class form between 2 or more classes. Its Identifiers, functionally depend on the identifiers of the classes from which it is form. But sometimes, an attribute can depend functionally on 2 identifiers, belonging to 2 different classes. An association class can also have associations with other classes.



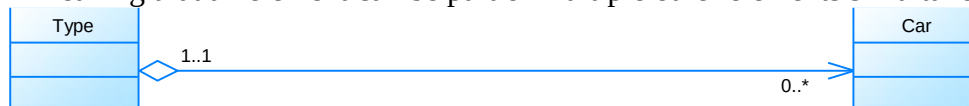
A student can have multiple enrolments for one and the same study program, which is not the intention. In contrast, a student can enrol for a specific study program only once. A student can only be granted an exam meeting to discuss the result of the student's written exam once.

vii. Aggregation:

In class diagrams we often use association that expresses relationships of container/content or set/elements. This relationship is represented by a small empty diamond on the site of the aggregate. UML differentiates between two types: **shared aggregation** and **strong aggregation/composition**. Both are represented by a diamond at the association end of the class that stands for the “whole”. The differentiation between composition and shared aggregation is indicated by a solid diamond for a composition and a hollow diamond for a shared aggregation. Both are transitive and asymmetric association. Transitivity means that if B is part of A and C is part of B, C is also part of A. Asymmetry expresses that it is not possible for A to be part of B and B to be part of A simultaneously.

a. Shared aggregation

A shared aggregation expresses a weak belonging of the parts to a whole, meaning that parts also exist independently of the whole. The multiplicity at the aggregating end may be greater than 1, meaning that an element can be part of multiple other elements simultaneously.



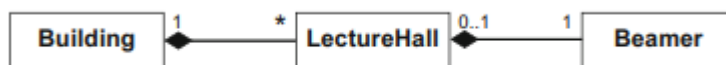
NB: An aggregated element can be link to other classes.

- Deleting the set does not delete the element.
- An aggregate can be multiple.
- Aggregation implies a relationship where the element can exist regardless of the aggregate
- Aggregation is considered a weak association

b. Composition

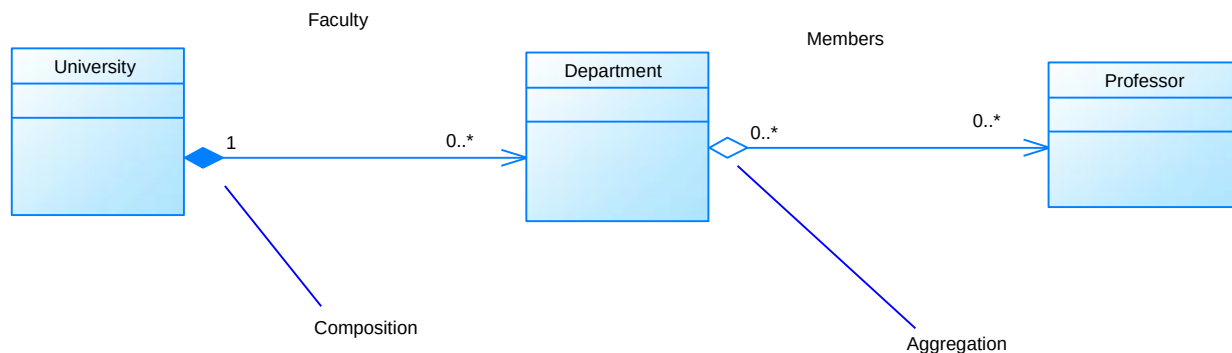
It is a special case of aggregation in which the life of the component is linked to that of the aggregate. It often refers to a physical capacity. In the composition association. The aggregate can't be multiple. In addition to the aggregation, the composition implies a coincidence of the lifetime of the component. The destruction of the aggregate (container) automatically implies the destruction of all the linked component. The composition is represented by a small black diamond beside the aggregate.

Example:



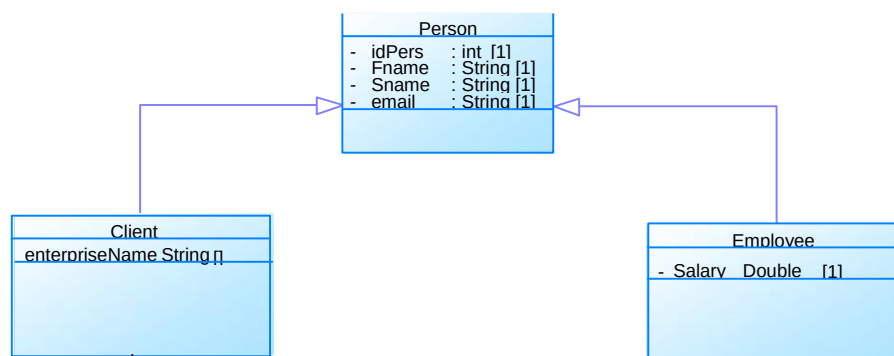
A lecture hall is part of a building. Due to the multiplicity 1, there is an existence dependency between elements of these two classes. The lecture hall cannot exist without the

building. If the building no longer exists, the lecture hall also does not exist anymore. The situation is different for a beamer which is also associated with a lecture hall by a composition. However, the multiplicity 0..1 is specified at the aggregating end. This means that the beamer can exist without the lecture hall, that is, it can be removed from the lecture hall. If the beamer is located in the lecture hall and the lecture hall ceases to exist—for example, because the building is torn down—the beamer also ceases to exist. However, if it was removed from the lecture hall beforehand, it continues to exist



viii. Generalisation/specialization

The principle of generalisation make it possible to identify among the object of a class subset of object with specific definitions. The more specific class (daughter class) is consistent with the more general class or main class (mother class) i.e. it inherit all attributes, members, relationships form the main class and can have other relationships. A generalisation relationship is indicated by an arrow pointing the parent class.

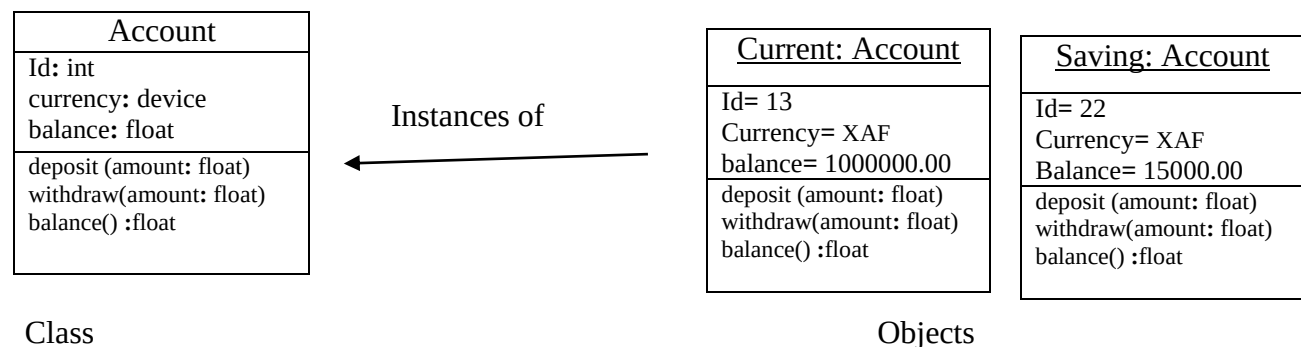


2. Object diagram

The object diagram represents a specific instance of a class diagram at a specific moment. It allows you to express the class diagram through examples in highlighting the imperceptible details in the class diagram. In other words, the class diagram models the rules, and the object diagram models the facts. One Object diagram is materialized as a class diagram but with some little differences. An object in a system can be an account in a virtual banking system. The object there is defined by his name that will represent him uniquely in memory, his state (his attributes values at a particular moment) and his behaviour (concerning the different task it can do). In an object diagram you don't need to put the last part concerning the operations and the multiplicities are not represented. An object is an instance of a class.

The purpose of an object diagram is similar to the one of class diagram. The difference is that the class diagram represents **an abstract model consisting of classes and their relationships**. However, an object diagram represents **an instance at a particular moment which is concrete in nature**. It means that the object diagram is closer to the actual system behaviour.

Example of a class with his objects:



3. Package diagram

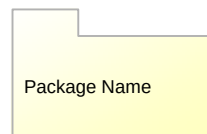
It allows the system to be broken down into more easily observable categories or parts called "packages". In fact, a package serves the same purpose as a folder on your computer. Packages can be built to represent physical or logical relationships. Package diagram helps to simplify complex class diagrams in a well-ordered visual form. A package diagram is effectively a class diagram that only shows packages.

Packages appear as rectangles with small tabs at the top; the Package name is on the tab or inside the rectangle. The dotted arrows are dependencies, one package depends on another if changes in the other can force changes in the former.

Depending on where a package is used, packages can participate in different types of relationships. For example, in a class diagram, packages represent groupings of classes. Therefore, aggregation and association relationships are possible.

a) Modelling package diagrams

- i. **Package:** it's used to group together logically related element within a system. Each element contained within the package should be packable elements and have a unique name.

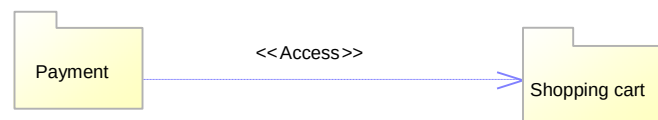


- ii. **Dependency:** it is a visual representation of how an element or a set of element depend on or influence another element.

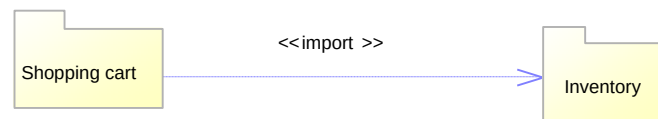
----->

Dependencies are divided into 2 groups: **the access and the import dependency.**

- *Access dependency:* it indicates that a package require assistance from the function of another package.



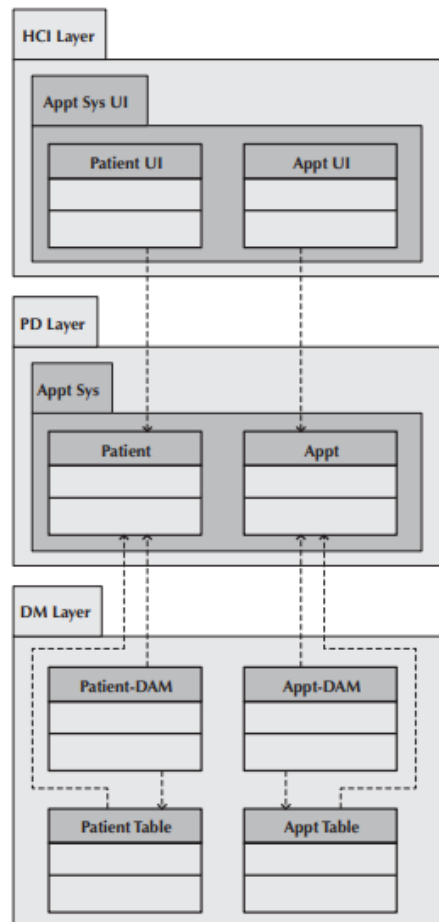
- *Import dependency:* it indicates that a functionality has been imported from one package to another



b) Guidelines for Creating Package Diagrams

- Use package diagrams to logically organize designs. Specifically, use packages to group classes together when there is an inheritance, aggregation, or composition relationship between them or when the classes form a collaboration.
- In some cases, inheritance, aggregation, or association relationships exist between packages. In those cases, for readability purposes, try to support inheritance relationships vertically, with the package containing the superclass being placed above the package containing the subclass. Use horizontal placement to support aggregation and association relationships, with the packages being placed side by side.

- When a dependency relationship exists on a diagram, it implies that there is at least one semantic relationship between elements of the two packages. The direction of the dependency is typically from the subclass to the superclass, from the whole to the part, and with contracts, from the client to the server. In other words, a subclass is dependent on the existence of a superclass, a whole is dependent upon its parts existing, and a client can't send a message to a non-existent server.



4. The component Diagram

It is mainly use to describe the dependencies between various software component such as the dependency between executable file and source files.

It represent the internal structure of your software (Software component). You must represent the different classes or libraries or modules or files. It represents the technical architecture of your system.

a Modelling component of a component diagram

- i. **A component** represents a software entity of a system. It is represented by a rectangular box with 2 rectangles protruding from the left side and considered as ports.



UML defines 5 component stereotypes:

1. << <<**Document** >> >>: it represents any document.
2. << <<**executable** >> >>: it is a program that can be executed
3. << <<**file**>> >>: it is a document having the source code
4. << <<**library** >> >>: It represent a static or dynamic library
5. << <<**Table** >> >>: it represents a relational database table.

ii. **Dependence**

It is use to model the relationship between 2 components. The notation use for the dependency relationship is a dotted arrow pointing from a given component to the component on which it depends



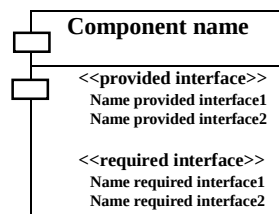
iii. **Interfaces**

Interfaces represent the places were the group of class in the component communicate with other system components. Two types of interfaces exists:

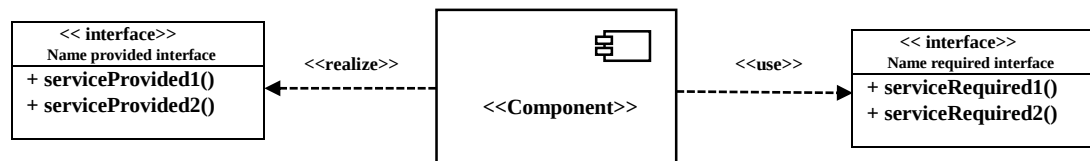
- Required interfaces: Interfaces providing services to the component needing them to be functioning. They are related to the component with the stereotype <<**use** >>
- Provided interfaces: Interfaces by which the component provides services himself to other components. They are related to the component with the stereotype <<**realize** >>

➤ **Different representations of interfaces:**

- ✦ Inside the component

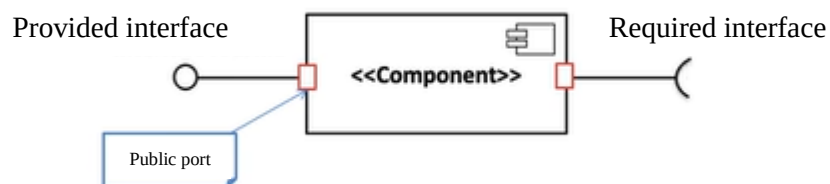


- ✦ Modelling with separated classes

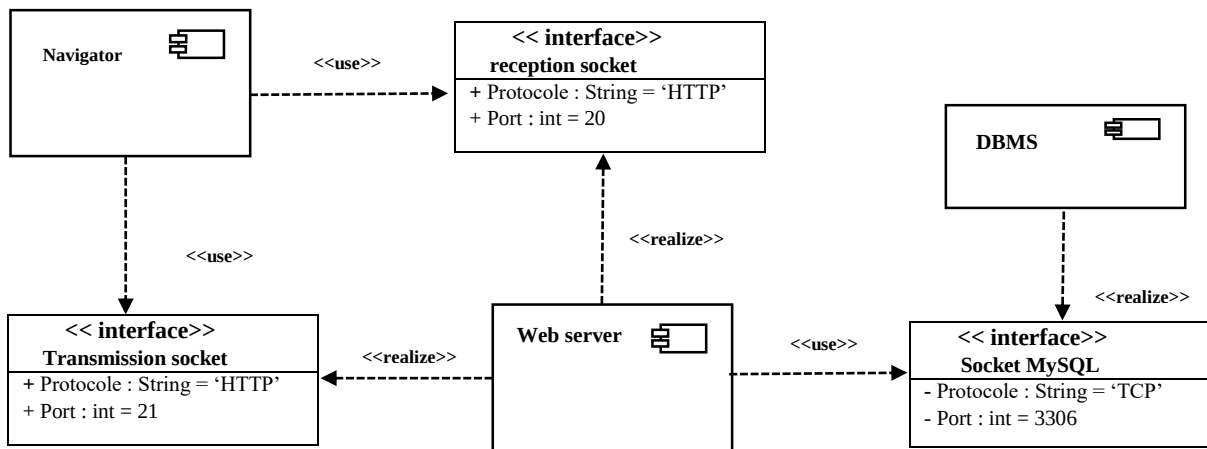


- ✦ Using Assembly connectors

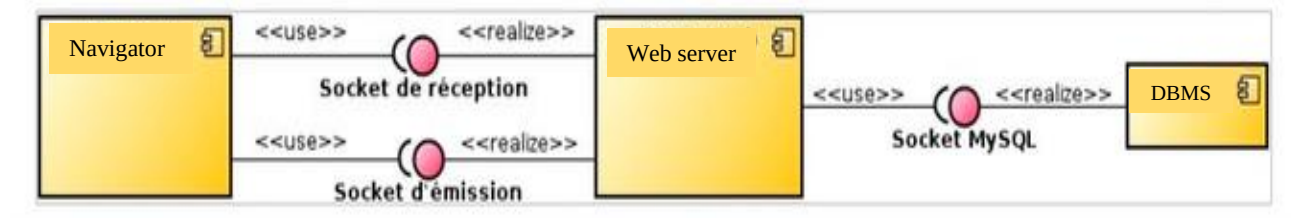
They determine how components can communicate together depending on which component requires which other to now provides information to other components



Example of a component diagram of an internet data transfer :



OR



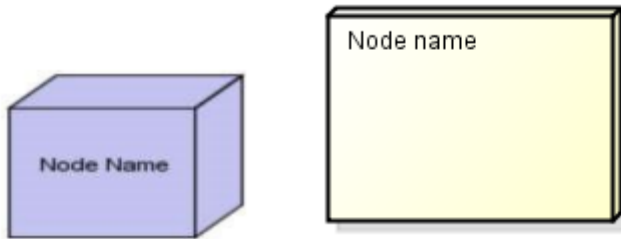
5. The Deployment Diagram

A deployment diagram is a visual representation of the physical architecture of a computer system. It's a diagram that represents the physical disposition of materials resources that constitute the system and shows the component repartition (software elements) that are executed inside these materials. It shows how the different hardware and software components of a system are connected and deployed on different nodes.

(For a web application we need a server that is physical and contains the application server that is software, your application will be deployed on the application server that will communicate with the web server. For other users to have access they will have a computer that is a material or hardware. That hardware is having a software that is the navigator.

a Modelling component of a deployment diagram

1. **Nodes:** Material or hardware resources are represented by a node



2. **Association:** it represent the link and the communication between nodes

Example of a deployment diagram of MessioAp:

The application architecture consists of the following resources:

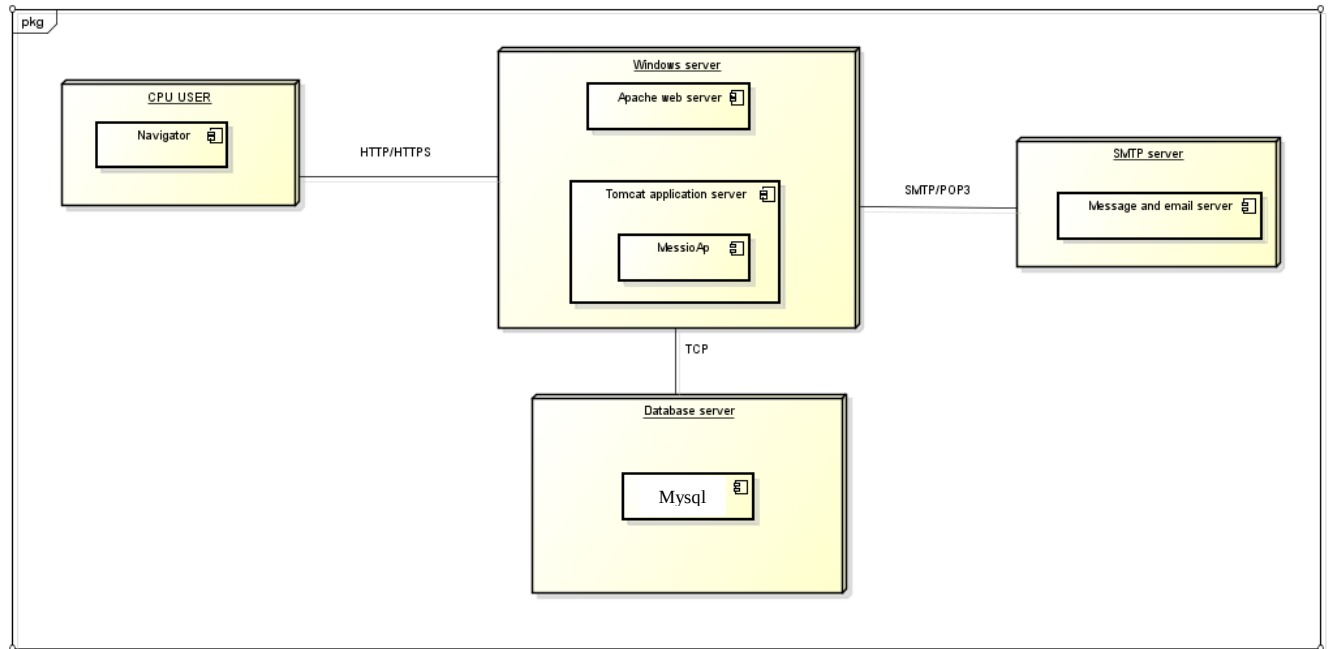
- Web client: It represents the web browsers with which the different collaborators connect to the application.
- WINDOWS server: It represents the server in which the application is deployed.

This server hosts both servers

- The Apache web server
- The Tomcat application server

The two servers are merged into the WINDOWS server and must be accessible to users on a LAN or WAN network.

- DBMS: MySQL database management system ensuring data storage
- SMTP server: It forwards the emails generated by the application to users when creating their profiles.



6. The profile Diagram